

Final Term Project: Real Estate Data

Ava Mohebbi

Department of Computer Science, Virginia Tech

CS 5805: Machine Learning I

Professor Reza Jafari

December 8th, 2024

Table of Contents

Abstract.....	3
Introduction.....	4
Dataset.....	4
Phase I: Feature Engineering and EDA	5
Phase II: Linear Regression	16
Phase III: Classification	20
Phase IV: Clustering and Association Rule Mining	34
Final Recommendations/Conclusion	40
References	41
Appendix	42

Table of Figures and Tables

Figure 1 <i>Boxplot For Numerical Features</i>	7
Figure 2 <i>State Counts.....</i>	8
Figure 3 <i>Status Counts.....</i>	8
Figure 4 <i>Random Forest Analysis</i>	10
Figure 5 <i>PCA Analysis</i>	11
Figure 6 <i>SVD Analysis.....</i>	12
Figure 7 <i>Covariance Heatmap</i>	14
Figure 8 <i>Correlation Heatmap.....</i>	15
Figure 9 <i>Linear Regression: Predicted Vs. Actual Values.....</i>	19
Figure 10 <i>Price Category Distribution Prior to Balancing.....</i>	21
Figure 11 <i>Decision Tree Confusion Matrices.....</i>	23
Figure 12 <i>Decision Tree ROC Curves.....</i>	23
Figure 13 <i>Logistic Regression Confusion Matrix</i>	24
Figure 14 <i>Logistic Regression ROC Curve</i>	25
Figure 15 <i>Elbow Method For K Selection</i>	26
Figure 16 <i>kNN Classifier Confusion Matrix.....</i>	26
Figure 17 <i>kNN Classifier ROC Curve.....</i>	27
Figure 18 <i>SVM Confusion Matrix</i>	28
Figure 19 <i>SVM ROC Curve</i>	28
Figure 20 <i>Naïve Bayes Confusion Matrix.....</i>	30
Figure 21 <i>Naïve Bayes ROC Curve</i>	30
Figure 22 <i>MLP Confusion Matrix.....</i>	32
Figure 23 <i>MLP ROC Curve.....</i>	32
Figure 24 <i>K-Means Clustering Silhouette Method</i>	35

Figure 25 <i>K-Means Clustering Elbow Method</i>	35
---	----

Table 1 Percentage of Missing Values Per Feature	5
Table 2 <i>VIF Table</i>	13
Table 3 <i>Backward Stepwise Regression</i>	17
Table 4 <i>Final Regression Model</i>	18
Table 5 <i>Confidence Interval Analysis</i>	18
Table 6 <i>Decision Tree Results</i>	22
Table 7 <i>Logistic Regression Results</i>	24
Table 8 <i>kNN Results</i>	26
Table 9 <i>SVM Results</i>	28
Table 10 <i>Naïve Bayes Results</i>	29
Table 11 <i>MLP Results</i>	31
Table 12 <i>Classification Summary Table</i>	33
Table 13 <i>K-Means Clustering Results (Silhouette Method)</i>	35
Table 14 <i>K-Means Clustering Results (Elbow Method)</i>	36
Table 15 <i>Apriori Results Sorted by Confidence</i>	38
Table 16 <i>Apriori Results Sorted by Lift</i>	39

Abstract

The US housing market has experienced an influx of demand causing housing prices to skyrocket in recent years. This has made real estate price prediction and pattern discovery critical for understanding market trends. This study utilized a dataset with over 2 million real estate listings with features such as price, size, and location to predict property prices and explore feature relationships using linear regression, classification, clustering, and association rule mining. The preprocessing steps included handling missing values, feature scaling, encoding, outlier detection, and feature selection. Linear regression was used to predict property prices but resulted in a high mean squared error and a low r-squared value, suggesting it was not suitable for this dataset. Classification models including multilayer perception (MLP), k-nearest neighbors (kNN) and logistic regression provided superior results, achieving ROC-AUC scores of 0.81, 0.80, 0.80, respectively. Clustering analysis using k-means and DBSCAN showed variations in groupings that were influenced by the value of k and algorithmic

differences. Finally, association rule mining uncovered interesting relationships such as the link between property sizes and seasonal trends. Overall, these findings show the need for advanced regression models for future studies (such as random forests or kNN regressors) and suggest promising results for classification methods.

Introduction

The US real estate market has experienced a significant surge in prices over the past decade, making the ability to predict property values increasingly important for various stakeholders including investors, buyers, realtors, and policymakers. Price predictions allow buyers and investors to make informed decisions when making offers to buy properties. Realtors benefit by gaining insight into market trends, allowing them to price homes competitively to ensure timely sales and maximum profits. Policymakers can also use this information to anticipate future costs of living and estimate revenues from real estate taxes. This study consists of four phases to explore machine learning applications in the real estate market: Phase 1 involves exploratory data analysis and feature engineering, Phase 2 focuses on linear regression techniques for price prediction, Phase 3 evaluates classification models for price categories and Phase 4 applies clustering and association rule mining to uncover patterns in the data. By utilizing these methods, the research aims to find the best models to provide actionable insights for improving decision making within the real estate industry.

Dataset

The original dataset consisted of 2,226,382 records of real estate listings in the US, though it was down sampled to 66,792 observations during the preprocessing phase to help with computational times. The dataset also included the following 12 features:

Categorical: Brokered_by, status, street, city, state, zip_code, prev_sold_date

Numerical: price, bed, bath, acre_lot, house_size

The original dataset was obtained from Kaggle and was originally sourced from listings available on Realtor.com (Sakib, 2022). For both regression and classification, price served as the dependent variable, since it was the value being predicted. All other features were independent variables since they were used to predict the price. This dataset is important in industry because these features can be used to predict property prices and provide insight into market trends which allows stakeholders to make more informed decisions and create strategies that are tailored to market needs.

Phase I: Feature Engineering and EDA

Data Cleaning

The initial step in data preprocessing involved cleaning the dataset to ensure its quality and reliability. The dataset was examined for duplicates, and it was confirmed that no duplicate records were present. Next, the dataset was analyzed for missing values, which is summarized in Table 1. To address the missing data, the proportion of NaN (not a number) values for each feature was calculated. The table indicates that none of the features had an excessive amount of missing information so to avoid the possibility of removing valuable information the missing values were imputed. The missing values for categorical features were replaced with the mode since it represents the most frequent category. For numerical features, the missing values were replaced with the median since it is less sensitive to outliers. This insured that extreme data points did not distort the imputed values.

Table 1

Percentage of Missing Values Per Feature

Feature	Percent Missing (%)
prev_sold_date	32.98
house_size	25.53
bath	22.99
bed	21.62
acre_lot	14.62

street	0.49
brokered_by	0.20
price	0.07
city	0.06
zip_code	0.01
state	0.00
status	0.00

Encoding/Feature Engineering

The categorical features needed to be encoded to be compatible with the machine learning algorithms. Status and state were low cardinality features; therefore, one hot encoding was implemented. This technique converts the categorical features into a binary representation with separate columns for each unique category. The first column was dropped to avoid the dummy variable trap, which could lead to multicollinearity. Brokered_by, zip_code, and city each had over 10,000 unique labels. Due to the computational inefficiency and sparsity issues that one hot encoding would cause; label encoding was used instead. This consisted of assigning a numerical value for each unique label. These label encoded features were normalized to prevent larger numbers from disproportionately influencing the model during training. Finally, the prev_sold_date feature was transformed by extracting the year and month as two separate columns. These new features would capture both seasonal patterns and long-term trends. To capture the chronological order of year, it was label encoded to make it compatible with the machine learning models. Because month does not inherently have an ordinal relationship (ex. February is not greater than January), one hot encoding was applied.

Down sampling

To address the large size of the dataset and reduce processing times, the data was down sampled to approximately 66,792 observations. These observations were selected in a way to maintain the balance and structure of the original dataset.

Data Visualization

Visualizations were created to explore and gain insights into the dataset. Box plots were generated for all the numeric features to understand the data's distribution and potential outliers. The box plots revealed that price had a wide distribution with numerous datapoints above the 75th percentile. Similarly, acre_lot had a wide spread, with many points in the upper quartile. Bar charts were created for the categorical features to understand the frequencies of different labels. California, Illinois, and Texas appeared most often in the dataset which aligns with their status as large states with major metropolitan areas. Finally, the status bar chart indicated that most homes in the dataset were “for sale” with very few “ready to build” homes.

Figure 1

Boxplot For Numerical Features

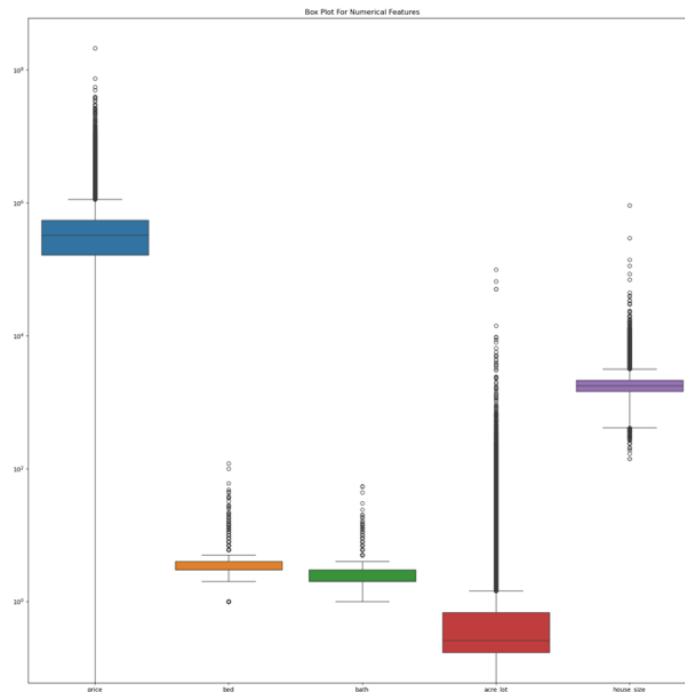


Figure 2
State Counts

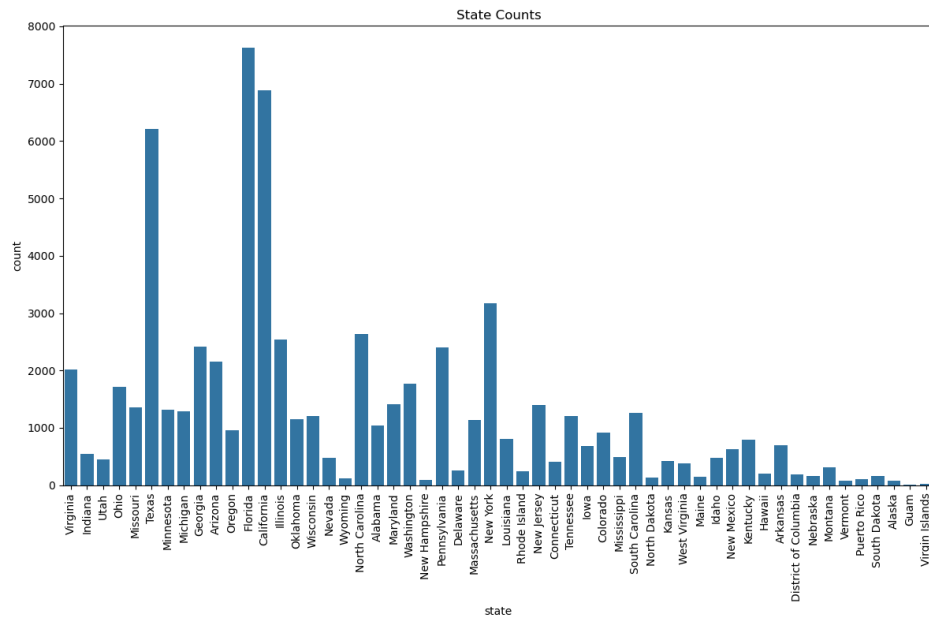
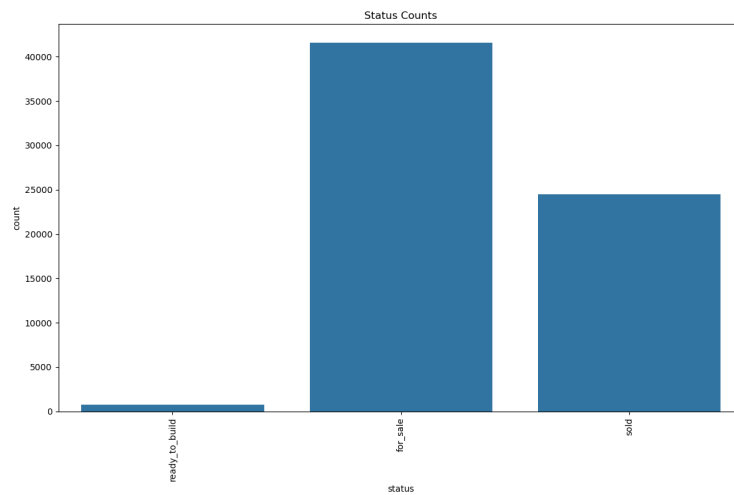


Figure 3
Status Counts



Scaling

Because many machine learning algorithms are sensitive to the scale of the data, the numerical features were standardized for optimal performance. This step is especially important for distance-based algorithms. Standardization was applied to the numerical features: bed, bath,

acre_lot, and house_size. Standardization transforms the data so that the mean becomes 0 and all other datapoints are within one standard deviation, which resulted in data points between the values of -1 to 1. As discussed previously, the categorical features that were label encoded were normalized.

Outlier Detection

Detecting and removing outliers was an important step to prevent misleading results. Local Outlier Factor (LOF), a density-based method, was used to identify outliers in the dataset. This method was selected because of its ability to handle complex relationships between features. An LOF score was assigned to each datapoint and all scores lower than -2.5 were considered outliers and removed (Sangole, 2024). This resulted in the removal of 8,750 outliers before implementing any machine learning algorithms. In real estate data, outliers often represent meaningful cases such as luxury homes or distressed properties. For this reason, a higher threshold was selected to minimize the risk of excluding valid yet extreme datapoints.

Feature Selection

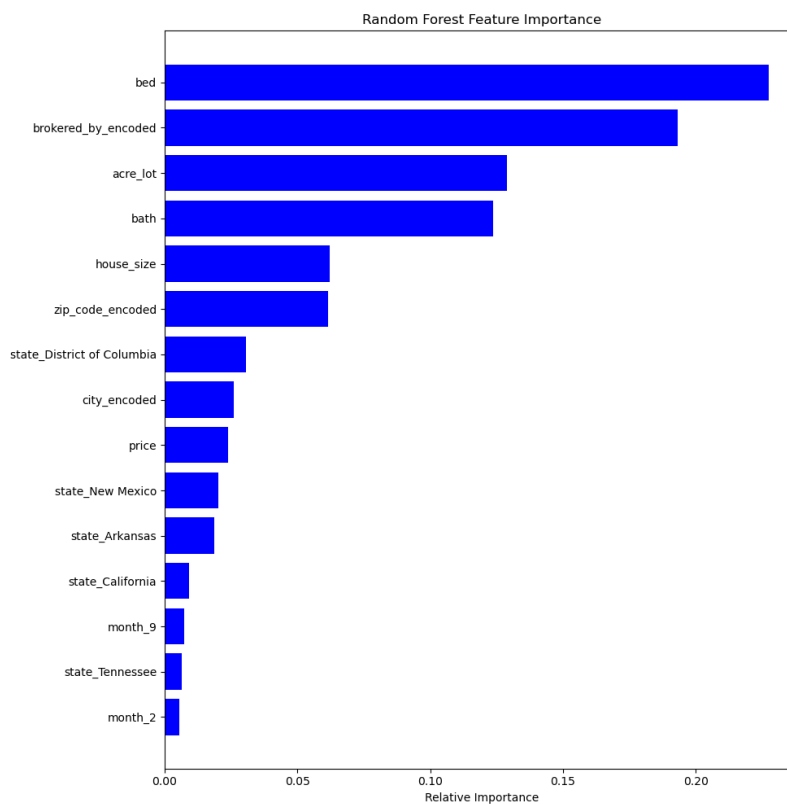
Feature selection is a critical step in machine learning that serves several purposes. By reducing the number of features, it allows for faster processing times and enhances the model's performance. This process can also mitigate overfitting of the data where the model performs well on the training set but poorly on the test set. The primary goal of feature selection is to keep the relevant features that provide predictive power while eliminating redundant or irrelevant ones. In this study, four feature selection methods were studied: Random Forest Analysis, Principal Component Analysis, Singular Value Decomposition, and Variance Inflation Factor.

1. Random Forest Analysis

Random Forest is an ensemble learning method that is known for its ability to handle large datasets and evaluate feature importance. This algorithm ranks the contributions of each feature in predicting the target variable as shown in Figure 4. According to the Random Forest results, `bed`, `brokered_by`, and `acre_lot`, were identified as the most important features in the dataset. This suggests that the agency listing the property has a significant role in determining its value. This is likely because brokers work with specific regions, or property types (ie. luxury or affordable housing). `Bed` and `acre_lot` highlight the impact of property and land size on property values. Larger lot sizes and bedroom counts are typically associated with greater property values.

Figure 4

Random Forest Analysis



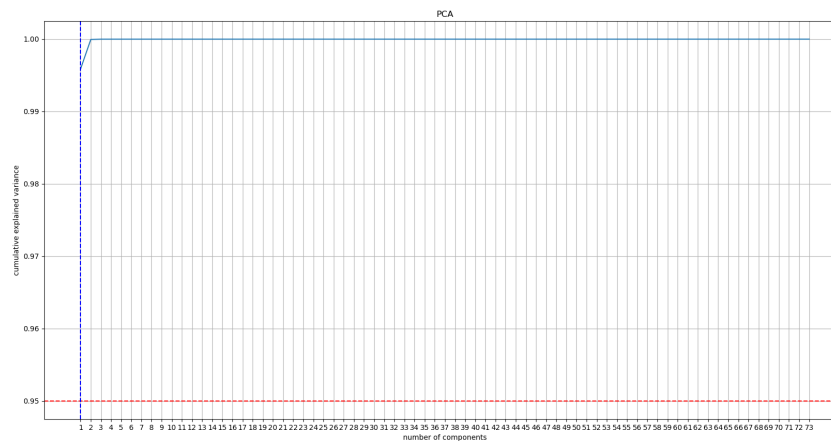
Note. Top 15 features shown for clarity.

2. Principal Component Analysis (PCA)

PCA is a linear dimensionality reduction technique that is used to simplify a dataset while maintaining features with high variance. The cumulative explained variance was plotted as shown in Figure 5 to assess the contribution for each component. The analysis showed that only one component explains over 95% of the variance, indicating that much of the information in the dataset can be captured in one dimension. To further assess multicollinearity, the condition number was computed to be 35,470,441,207. A high magnitude like this suggests there is significant multicollinearity or redundancy among the features. This could make machine learning models less stable and indicated that it was important to identify and remove features contributing to multicollinearity.

Figure 5

PCA Analysis



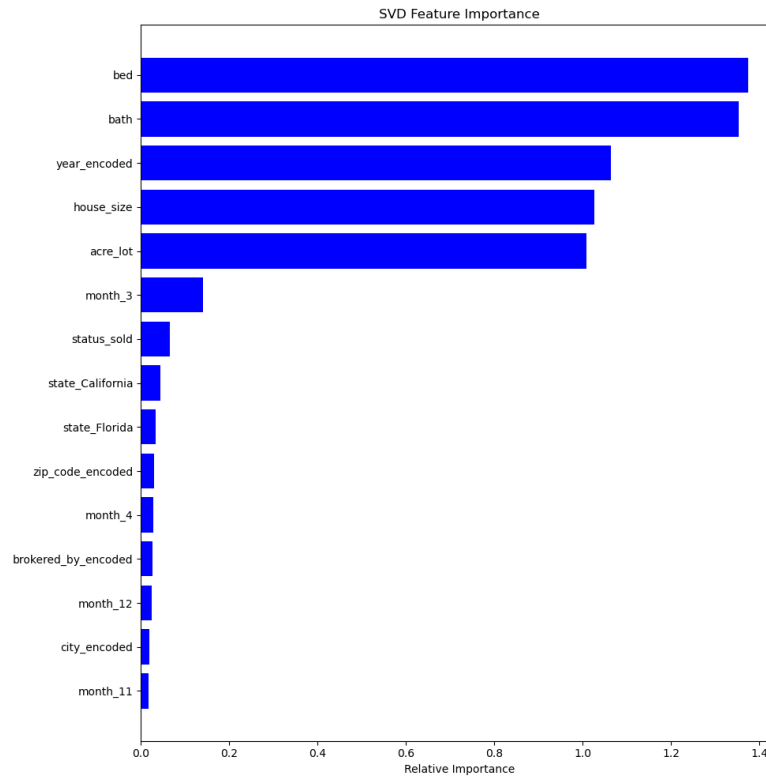
3. Singular Value Decomposition (SVD)

SVD is another technique to reduce the dimensionality of the dataset while maintaining key patterns and information in the data. It considers the contribution of each feature to the

data's variance. According to Figure 6, the features with high importance are bed, bath, year_encoded, house_size, and acre_lot.

Figure 6

SVD Analysis



Note. Top 15 features shown for clarity.

4. Variance Inflation Factor (VIF)

VIF is a tool to measure the severity of multicollinearity in regression. For this study, a VIF above 5 was considered high, and VIF above 10 was considered severe. All features with a VIF above 5 were flagged for further analysis and displayed in Table 2. The analysis revealed 17 features exceeding this threshold. Zip_code_encoded had the highest VIF with a value of 380. The high multicollinearity of zip code could be attributed to its strong relationship with other geographic features such as city and state. Bed, bath, and a few of the states also had high VIF

values, though not nearly as high as zip code. This indicated that these features should be removed from the dataset to help with interpretability.

Table 2

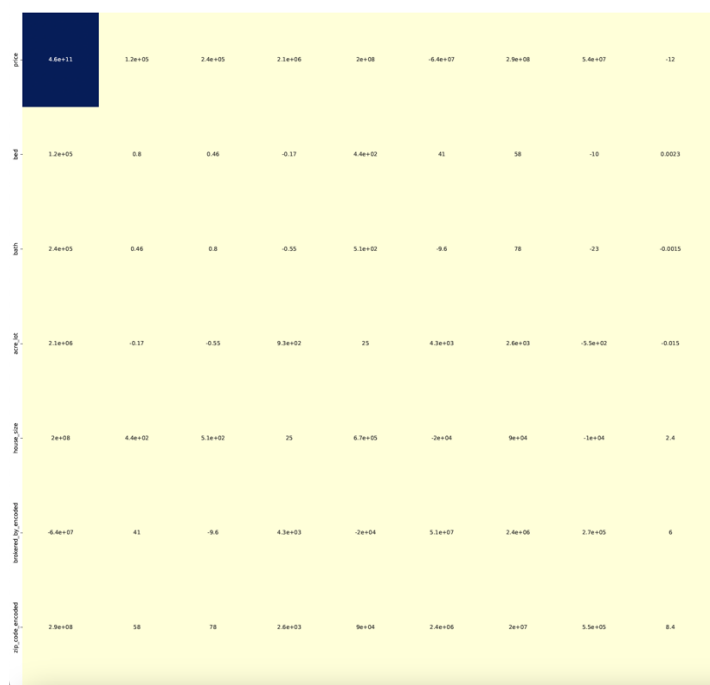
VIF Table

Feature	VIF
zip_code_encoded	380.481
year_encoded	112.262
state_California	50.671
state_Texas	29.691
state_Washington	15.989
state_Arizona	14.670
bed	12.941
bath	11.641
state_Oregon	9.478
month_3	8.599
house_size	8.470
state_Illinois	8.106
state_Florida	6.873
state_Colorado	5.998
state_Oklahoma	5.752
state_Missouri	5.285
state_New Mexico	5.267

Covariance

Figure 7 shows the covariance heatmap which provides insights into the relationships and variance among the features of the dataset. The off diagonal shows the covariance between variables, indicating the strength of their relationship. The main diagonal shows the variance for each feature and reflects how spread out the data is from the mean. The heatmap uses a color scale to highlight the magnitude of the values in the matrix. The variance for price was the main value that stands out in the heatmap, indicating it has a very wide distribution compared to other features. This means there is a wide range of prices across the dataset, which could potentially create challenges in predicted prices. A result like this is expected in real estate data due to the wide variety of property types (ex. luxury, affordable, etc.).

Figure 7
Covariance Heatmap



Note. This only provides a portion of the covariance heatmap for clarity.

Correlation

A Pearson correlation matrix shows the strength and direction between pairs of variables in the dataset. The correlation coefficient ranges between -1 to 1, where negative values indicate a negative correlation, positive values indicate a positive correlation and a value of 0 signifies there is no relationship. In this analysis there were a few variables that stood out with high correlation coefficients. Coefficients above 0.50 were identified for further analysis. The strongest correlation was observed between house size and bath (0.67), followed by house size and bed (0.62). The third highest correlation coefficient was between bed and bath with a value of 0.58. These relationships are intuitive since larger houses typically have more bedrooms and bathrooms. These relatively strong correlations indicate potential redundancy among features which could affect the model's performance.

Figure 8*Correlation Heatmap*

Note. This only provides a portion of the correlation heatmap for clarity.

Final Feature Selection

The results of the previous feature selection methods were used to determine which features to retain and exclude. High multicollinearity was significant in the dataset and was addressed by removing features with high VIF. Specifically, `zip_code_encoded` exhibited severely high VIF and relatively low importance so it was excluded. Similarly, because the states California, Texas, Washington, and Arizona had very high VIF values (above 10), they were excluded. Other states with high VIF values (between 5-10), including Oregon, Illinois, Florida, Colorado, Oklahoma, and Missouri, were also excluded. The correlation heat map indicated high correlation between the number of beds and baths so bath was removed to reduce redundancy. `House_size` was also excluded because of its high correlation with beds and baths and its high VIF. Key features like `brokered_by`, `acre_lot`, and `bed` were of identified

as highly important through Random forest and SVD analyses and were retained. The months, city, and status were shown to have some importance, so these were also kept. Random forest analysis and SVD indicated that many of the states were not important to predicting house prices, some with values near 0, so these states were removed from the dataset. Finally, street was removed from the dataset due to its high cardinality, which resulted in a large and sparse matrix that increased computational complexity without adding predictive power. Broader geographical features such as city and states were retained since they provided predictive power at a less granular level.

Phase II: Linear Regression

The objective for Phase II was to predict property prices using linear regression. After preprocessing the data and addressing multicollinearity, the data was split into 80% training set and 20% testing set. The numerical features were scaled after the dataset was split to avoid data leakage and the statsmodels package was used to implement backward stepwise regression and analyze confidence intervals and F-statistics (Seabold, 2010).

Backward Stepwise Regression/T-Test Analysis

Backward stepwise regression was implemented to remove statistically insignificant variables. A model was fitted and then the variable with the highest p-value was dropped. This process was repeated until all remaining p-values were below 0.05. Table 3 shows the results of this approach, which resulted in the removal of 13 features.

To evaluate the model performance, various metrics were used. The Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) are statistical measures to compare model performance. Both AIC and BIC penalize a model for complexity, but BIC applies a stronger penalty. Lower AIC and BIC indicate a better fitting model. Although these values slightly improved as insignificant features were removed, they remained high, suggesting

that the model could be further improved. The R-Squared and adjusted R-squared which explain the proportion of variance that is explained by the independent variables did not change as the insignificant features were eliminated. This indicates that these features most likely did not add explanatory power to the model.

Final Linear Regression Model

The final regression model was generated with all the insignificant features dropped and the results are displayed in Table 4. The R-Squared indicates the proportion of variance in the dependent variable that is explained by the independent variables. In this case, only 23% of the price is explained by the model, suggesting there is room for improvement and the model does not provide significant explanatory power. The Adjusted R-Squared is a similar metric but accounts for the number of features and penalizes irrelevant features. The table indicates that the R-squared and adjusted R-squared are the same, meaning the dropped features likely did not contribute explanatory power to the model. Mean Squared Error (MSE) which measures the average distance between the predicted value and actual value was high. This indicated that the model's predictions were far from the actual values. Figure 9 shows the actual values versus the predicted values for both the train and test set with the reference line $y = x$. The plot illustrates that the predicted values deviate from the actual values as most datapoints were clustered at the base, and others were dispersed throughout the plot. The results indicate that linear regression may not be the most suitable model for this dataset. To improve the performance, additional predictive features may be required.

Table 3

Backward Stepwise Regression

Step	AIC	BIC	Adjusted R-Square	Feature with Max. p-value [DROP]	p-value
1	1410915.67	1411309.23	0.23	state_Guam	0.73
2	1410913.79	1411298.6	0.23	month_6	0.72

3	1410911.91	1411287.98	0.23	month_12	0.6
4	1410910.18	1411277.5	0.23	month_11	0.62
5	1410908.42	1411267.0	0.23	state_Virgin Islands	0.58
6	1410906.74	1411256.57	0.23	month_7	0.53
7	1410905.14	1411246.22	0.23	month_2	0.51
8	1410903.57	1411235.91	0.23	state_New Hampshire	0.45
9	1410902.14	1411225.74	0.23	month_9	0.11
10	1410902.7	1411217.54	0.23	state_Montana	0.05
11	1410904.41	1411210.51	0.23	month_4	0.05
12	1410906.23	1411203.59	0.23	month_8	0.05
13	1410908.07	1411196.68	0.23	month_5	0.04

Table 4*Final Regression Model*

Final Regression Model Metrics	
Metric	Value
AIC	1410915.67
BIC	1411309.23
R-Square	0.231
Adjusted R-Square	0.232
MSE	694,452,055,582.78

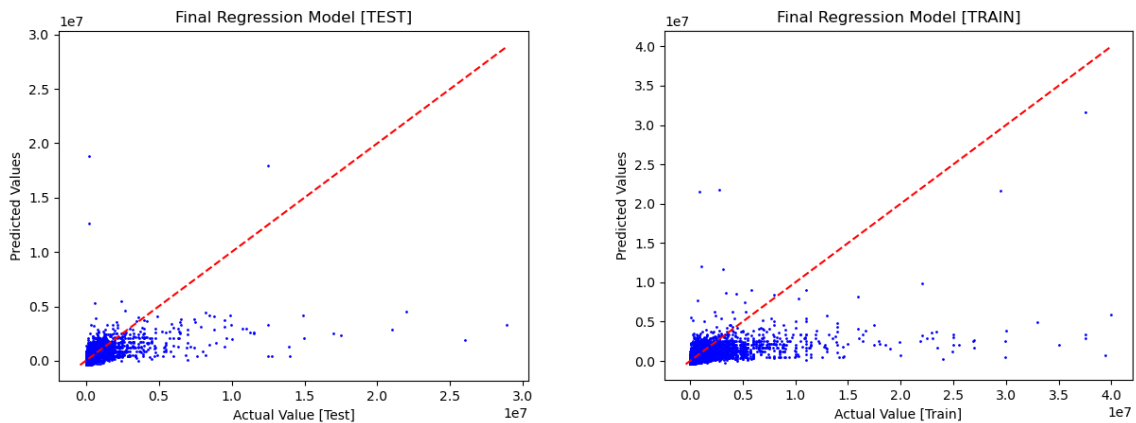
Table 5*Confidence Interval Analysis*

Feature	Lower Bound	Upper Bound
const	595438.043	597185.508
bath	449696.007	450258.644
acre_lot	223101.338	223660.165
brokered_by_encoded	-68413.066	-66419.018
city_encoded	38472.31	40443.444
status_ready_to_build	125584.097	131955.197
status_sold	-57687.179	-56298.846
state_Alaska	-274413.159	-258453.864
state_Arkansas	-270533.155	-264976.395
state_Georgia	-317210.293	-314213.225
state_Guam	-136612.336	-94077.892
state_Indiana	-262720.579	-256576.734
state_Iowa	-271753.406	-266282.736
state_Kentucky	-278893.78	-273784.728
state_Louisiana	-265157.782	-260036.088
state_Maryland	-286628.203	-282697.802

state_Massachusetts	147966.331	152259.451
state_Michigan	-237122.524	-233034.882
state_Minnesota	-194653.406	-190671.879
state_Mississippi	-324239.133	-317631.553
state_Montana	119701.318	127714.115
state_Nebraska	-336273.848	-325021.32
state_New Hampshire	-94849.014	-80231.729
state_New Jersey	-70742.469	-66901.762
state_New Mexico	-232173.329	-226523.494
state_New York	343128.225	345811.826
state_North Carolina	-258830.99	-255953.653
state_Ohio	-230405.215	-226816.679
state_Pennsylvania	-220381.228	-217372.567
state_South Carolina	-276536.48	-272303.335
state_Tennessee	-179966.534	-175793.741
state_Virgin Islands	-127718.839	-102057.16
state_Virginia	-224041.828	-220636.457
state_West Virginia	-289096.233	-281636.959
state_Wisconsin	-218541.681	-214431.964
month_2	13203.98	15543.533
month_4	32165.43	34117.21
month_5	57884.962	61118.898
month_6	-11517.341	-8066.764
month_7	-19219.773	-15641.095
month_8	57992.609	61597.554
month_9	45765.653	49534.32
month_10	62533.767	65813.571
month_11	9259.162	11568.269
month_12	7512.719	9688.975

Figure 9

Linear Regression: Predicted Vs. Actual Values



Confidence Interval Analysis

The confidence intervals for the coefficients are displayed in Table 5 and indicate that the true coefficient values will likely fall in this range with 95% confidence. A wider confidence interval indicates more uncertainty of the true value. The independent variables bath and acre_lot had a narrow interval, implying they have less uncertainty, while other features such as the months had wider ranges.

F-Test Statistics

The F-test assessed the overall significance of the regression model. The final F-statistic was 381 and the p-value was less than 0.05, meaning the null hypothesis could be rejected and the model was significant.

Phase III: Classification

Phase III consisted of applying various machine learning classifiers to determine the best performing classification technique for predicting price categories. The objective of this phase was to classify a property as having a low or high price. Discretization, data balancing and hyperparameter tuning were implemented. In total, seven classification algorithms were tested, and the final performance results were analyzed to determine the most effective strategy for classifying house price categories. The train and test accuracy was evaluated for each model to ensure the models weren't over or underfitted. The scikit-learn library was used to generate the machine learning models (Pedregosa et al., 2011).

Discretization

Since price was originally a continuous variable, it was necessary to discretize it for classification. A new column was added categorizing house prices into "low price" or "high price". Prices above the median were considered "high" while prices below this threshold were

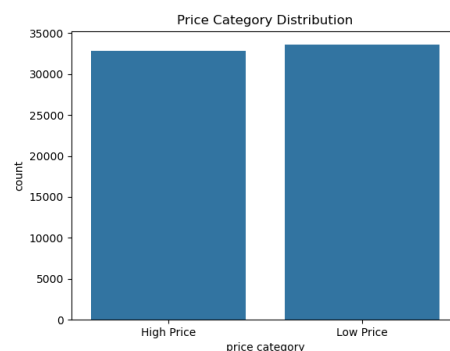
considered “low.” To make these categories compatible with the classifiers, the labels were encoded so that “low price” was relabeled as 0 and “high price” was relabeled as 1.

Data Balancing

The price category distribution was plotted to determine if the target variable (price category) was imbalanced. Figure 10 shows that the number of high and low prices is very similar, although there were a some more properties with a low price. To ensure there was full balance and to avoid any potential bias, the dataset was split into 80% for training and 20% for testing and then Synthetic Minority Oversampling Technique (SMOTE) was used on the training set. After balancing, both expensive and not expensive had 26,904 instances.

Figure 10

Price Category Distribution Prior to Balancing



Hyperparameter Tuning

To optimize the classifiers, grid search was used to identify the best hyperparameters. Grid search is an essential method for hyperparameter tuning as it searches all possible combinations of parameters. For this study, stratified 5-fold cross-validation was used with the grid search to ensure reliability of the results. This technique divides the set into five equal folds and for each iteration one-fold is used as the test set while the others are used for training. Stratification was implemented to maintain the balance of the target variable (low price and high price) across all folds.

1. Decision Tree Classifier

The decision tree classifier is a supervised machine learning algorithm that uses metrics like Gini impurity or entropy to split the data into subsets. The process is continued until a stopping criterion is met. Decision trees are easy to understand and interpret but are prone to overfitting. To address this, both prepruning and postpruning were employed to improve the model's generalizability. Prepruning limits the trees growth while its being constructed. Parameters like `max_depth` and `min_samples_split` were used to restrict tree complexity. Post pruning was applied to trim the tree after its fully grown where `ccp_alpha` was used to simplify the model and enhance performance.

Both Gini impurity and information gain were evaluated during grid search and the splitter was configured with both best and random. The `max_features` parameter, which determines the number of features at each split, was tested with `sqrt` and `log2`. For prepruning, the maximum depth was tested with values of 1,2,3,4 and 5 to reduce the tree complexity. Additionally, the `min_samples_split` parameter was tested with values of 20, 30 and 40. Lastly, for postpruning the cost complexity pruning parameter (`ccp_alpha`) was fine-tuned by testing alpha values between 0 to 1 with 100 equal intervals during grid search. The final results are displayed in Table 6. The accuracy results indicate that not implementing a pruning strategy leads to overfitting as indicated by the high train accuracy and low test accuracy.

Table 6

Decision Tree Results

Classifier	Precision	Recall	Specificity	F1-Score	AUC
Decision Tree (No Pruning)	0.65	0.69	0.682	0.66	0.66
Decision Tree (Pre-Pruning)	0.83	0.51	0.89	0.63	0.70
Decision Tree (Post-Pruning)	0.85	0.47	0.91	0.61	0.71

	Train Accuracy	Test Accuracy
No Pruning	0.999	0.657
Pre-pruning	0.707	0.703
Post-pruning	0.700	0.800

Best Parameters (Pre-Pruning): {'criterion': 'gini', 'max_depth': 2, 'max_features': 'log2', 'min_samples_leaf': 10, 'min_samples_split': 20, 'splitter': 'best'}

Best Parameters (Post-Pruning): {'ccp_alpha': 0.010101010101010102, 'criterion': 'gini', 'max_features': 'sqrt', 'splitter': 'best'}

Figure 11

Decision Tree Confusion Matrices

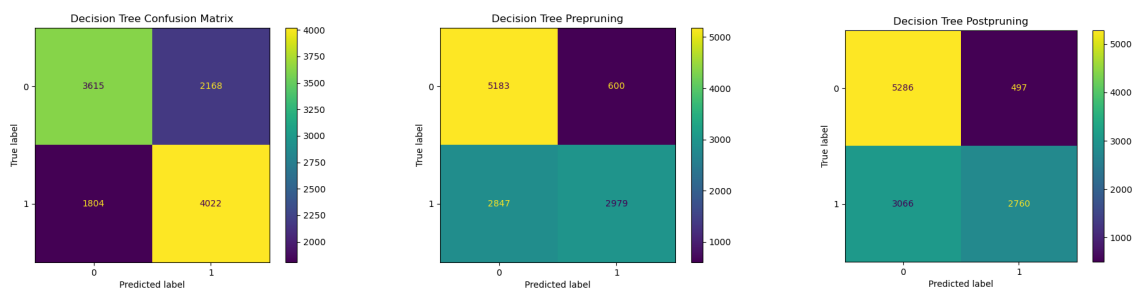
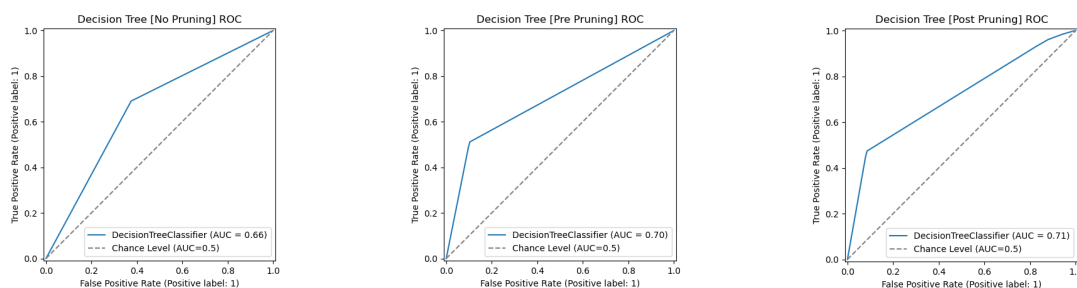


Figure 12

Decision Tree ROC Curves



2. Logistic Regression

Logistic regression is a supervised machine learning algorithm for binary classification. It predicts the probability of a class label based on the sigmoid function. The following parameters were tuned with grid search: solver, penalty, and C. Solver is the optimization algorithm used to

minimize the log loss function during training. For this study liblinear, and saga were evaluated using grid search. Penalty determines the type of regularization that is applied to the logistic regression model and can help overfitting. Both L1 (lasso) and L2 (ridge) regularization were tested for this study. Additionally, the hyperparameter C was used to determine the penalty strength, where values of 1, 10, 100, and 1000 were tested to evaluate their impact on model performance.

Table 7

Logistic Regression Results

Classifier	Precision	Recall	Specificity	F1-Score	AUC
Logistic Regression	0.74	0.68	0.76	0.71	0.80

Train Accuracy	Test Accuracy
0.718	0.719

Best Parameters: {'C': 10, 'penalty': 'l2', 'solver': 'saga'}

Figure 13

Logistic Regression Confusion Matrix

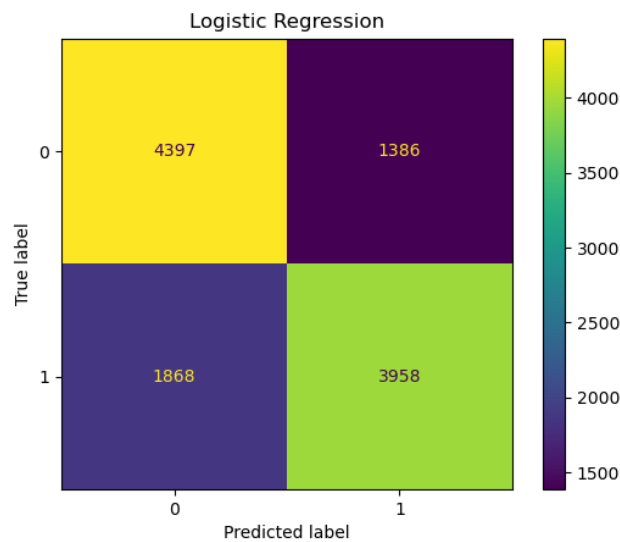
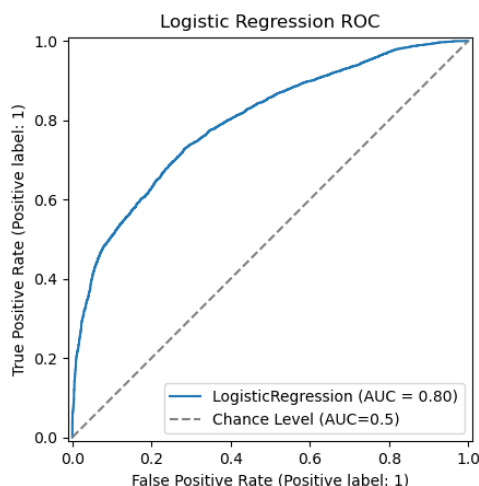


Figure 14*Logistic Regression ROC Curve*

3. *K-Nearest Neighbor (kNN)*

K-Nearest Neighbors is a lazy learning algorithm that relies on the proximity of data points to make predictions. It looks for data points within a predetermined number of nearest neighbors (k) and assigns the data point to the majority class of its neighbors. It's very sensitive to scaling since it relies on distance metrics, therefore it was important to ensure the data was properly scaled prior to implementing the algorithm.

Finding the optimal value for k is also critical. The elbow method was used to determine the optimal k value. This consisted of fitting the model for values of k between 1 - 40. For every k value, the error rate was calculated and plotted. Then, the plot was examined to find the "elbow point", which is where the error rate first starts to diminish. For the dataset, the optimal k value was determined to be 19, as it corresponded to the lowest error rate. This k value was then used during grid search for hyperparameter tuning.

The following parameters were tuned with grid search: leaf size, metric, p , and weights. Leaf sizes of 20, 40 and 1 were tested during grid search. The metric specifies the method used to calculate distances between data points; both the Minkowski and Chebyshev metrics were

tested. P is the power parameter for the Minkowski distance, both $p=1$ and $p=2$ were tested during grid search. Finally, weights refer to the influence each neighbor has on the classification decision and both uniform and distance were tested.

Figure 15

Elbow Method For K Selection

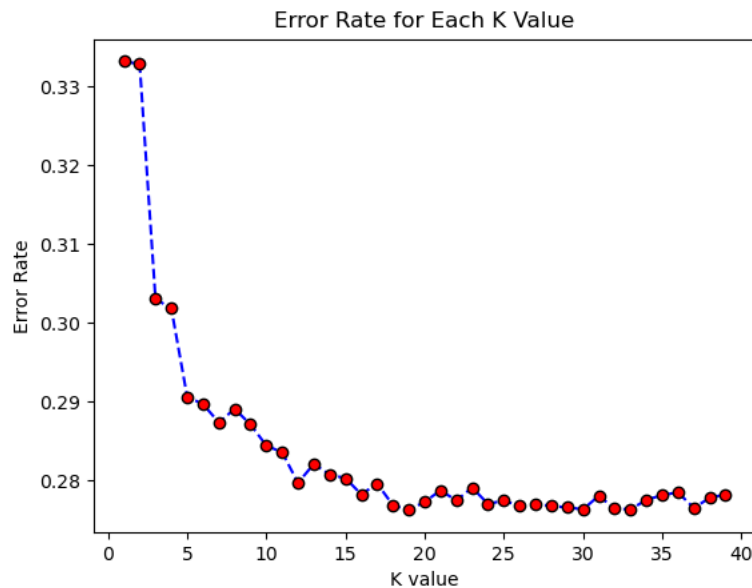


Table 8

kNN Results

Classifier	Precision	Recall	Specificity	F1-Score	AUC
kNN	0.74	0.68	0.76	0.71	0.80

Train Accuracy	Test Accuracy
0.754	0.723

Best Parameters: {'leaf_size': 20, 'metric': 'minkowski', 'p': 1, 'weights': 'uniform'}

Figure 16

kNN Classifier Confusion Matrix

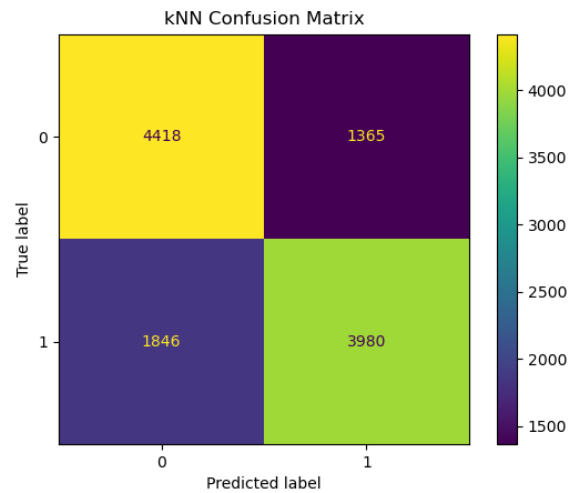
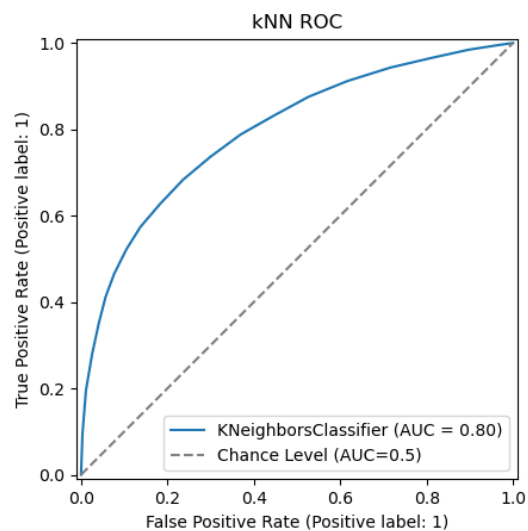


Figure 17

kNN Classifier ROC Curve



4. Support Vector Machine (SVM)

SVM is a supervised algorithm that classifies data by finding the optimal hyperplane in such a way that maximizes the distance between classes. Through the use of the kernel trick, SVMs can map data into higher dimensional spaces allowing it to handle nonlinearly separable datasets effectively. During grid search, various kernel types, including Linear, RBF, and

Polynomial, were evaluated. Additionally, the degree parameter, which determines the complexity of the polynomial kernel, was tested with values of 2 and 3.

Table 9

SVM Results

Classifier	Precision	Recall	Specificity	F1-Score	AUC
SVM	0.800	0.590	0.851	0.679	0.80

Train Accuracy	Test Accuracy
0.725	0.720

Best Parameters: {'degree': 2, 'kernel': 'rbf'}

Figure 18

SVM Confusion Matrix

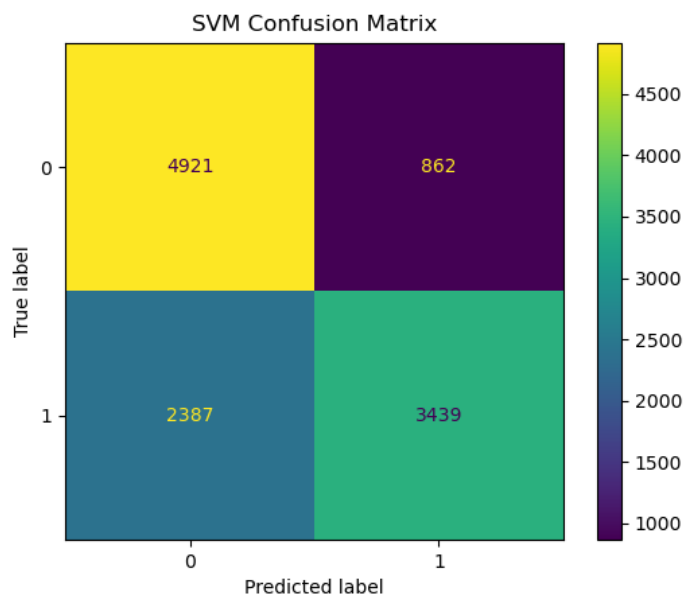
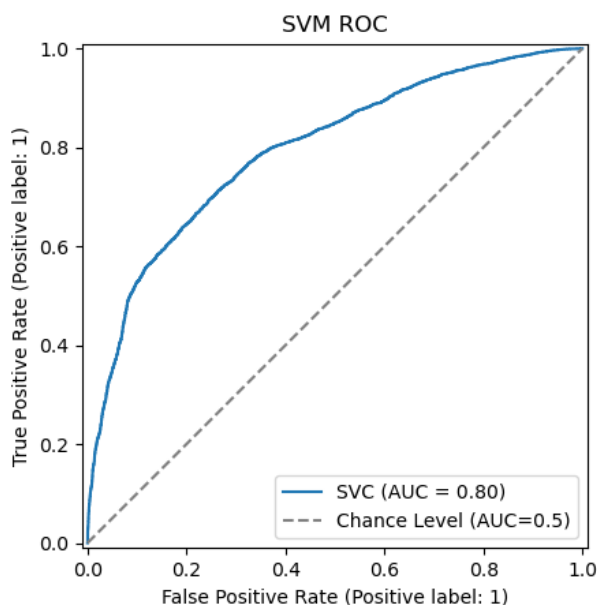


Figure 19

SVM ROC Curve



5. Naïve Bayes

Naïve Bayes is a supervised machine learning algorithm that assumes that the variables are conditionally independent of each other given the target class. While this assumption rarely holds true in real world scenarios, it remains popular because of its simplicity and speed for various tasks. There are several types of naïve bayes classifiers but the gaussian Naïve Bayes classifier was used for this study. The primary hyperparameter that was tuned was variance smoothing. Variance smoothing adds a constant to the variance estimates to make more stable models. The following values were tested for smoothing: 1e-9, 1e-8, 1e-7, 1e-6, 1e-5.

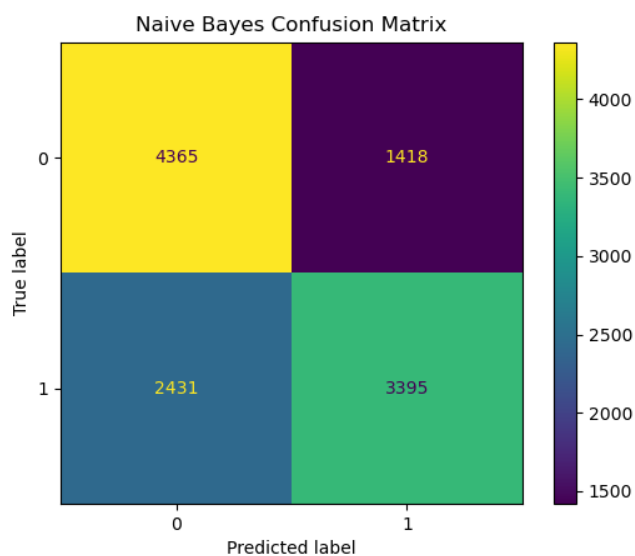
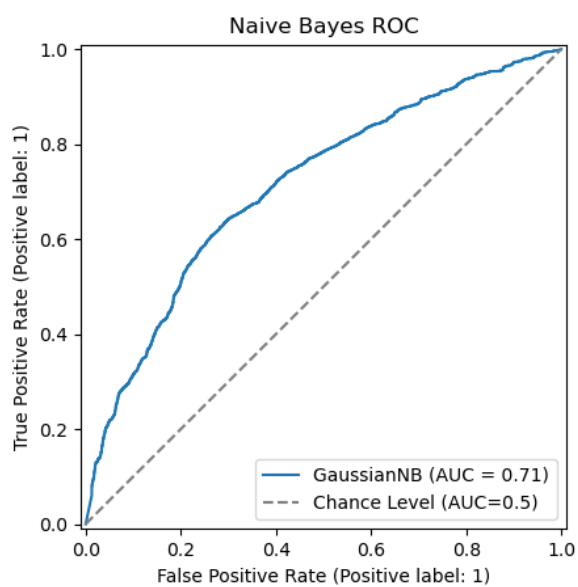
Table 10

Naïve Bayes Results

Classifier	Precision	Recall	Specificity	F-Score	AUC
Naïve Bayes	0.71	0.58	0.75	0.64	0.71

Train Accuracy	Test Accuracy
0.667	0.668

Best Parameters: {'var_smoothing': 1e-07}

Figure 20*Naïve Bayes Confusion Matrix***Figure 21***Naïve Bayes ROC Curve*

7. Neural Network (MLP)

A multi-layer perceptron (MLP) is a type of neural network designed to learn complex data patterns through interconnected layers. It consists of an input layer, hidden layers, and an output layer. For this study, the MLP classifier was used, and grid search was implemented to find the best architecture, optimization algorithms, and activation functions. Determining the number of layers and neurons needed for optimal performance involves some trial and error since increasing complexity can lead to overfitting and high computational cost. A few different architectures were tested including single layer, two-layer, and three-layer networks. Additionally, a few neuron configurations were explored: one scenario where the number of neurons was increased for each subsequent layer and another where the number of neurons decreased. The hidden layer sizes tested included configurations such as (62,), (62, 31), (62, 40), (62, 155), (62, 124), (62, 40, 20), and (62, 124, 62). The solvers Adam and SGD were tested as optimization algorithms and the activation functions Tanh and ReLU were also tested through grid search.

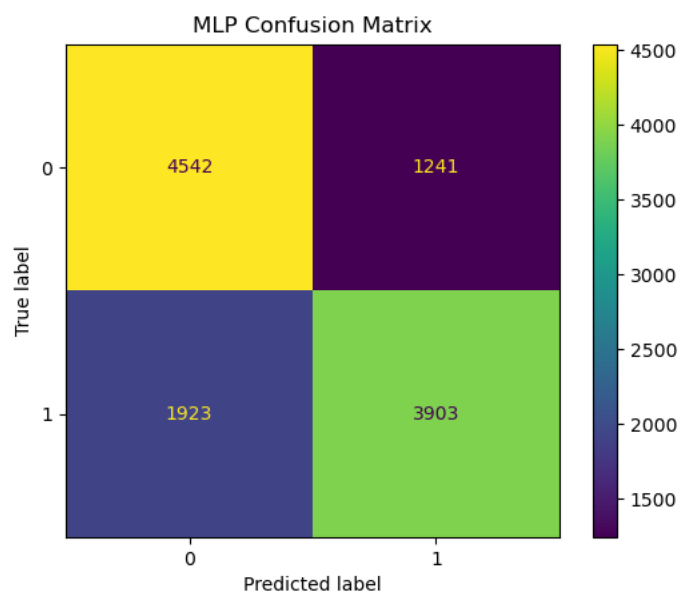
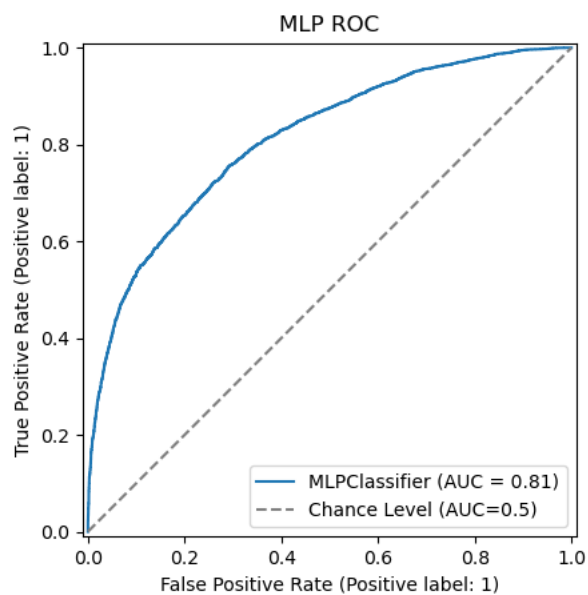
Table 11

MLP Results

Classifier	Precision	Recall	Specificity	F-Score	AUC
MLP	0.76	0.67	0.79	0.71	0.81

Train Accuracy	Test Accuracy
0.74	0.73

Best Parameters: {'activation': 'relu', 'hidden_layer_sizes': (62,), 'solver': 'adam'}

Figure 22*MLP Confusion Matrix***Figure 23***MLP ROC Curve*

Classifier Recommendation

Table 12 summarizes the performance of the classification models evaluated in this study. Metrics such as precision, sensitivity, specificity, F1-Score, and ROC-AUC were used to evaluate the models, each offering unique insights into the performance. Precision shows the proportion of positive cases that were correctly predicted, while specificity shows the proportion of negative cases that were correctly predicted. Recall represents the true positives divided by the total number of true positives and false negatives. There is a tradeoff between precision and recall and understanding which to prioritize depends on the context of the problem.

The F1-score is the harmonic mean of precision and recall and offers a balanced view of the how the model performs in terms of both false positives and false negatives. For this study, the F1-score was prioritized as the primary evaluation metric complemented by the ROC-AUC which measures the model's ability to distinguish between classes.

Among all the tested models, the multilayer perceptron offered the highest F1-score and highest ROC-AUC values and is the recommended classifier for this problem. Additionally, its precision, recall, and specificity ranged from 0.72-0.75, demonstrating strong and consistent performance across various metrics. The accuracy of the train and test set also indicated that the model did not suffer from overfitting. The K-nearest neighbor classifier and Logistic Regression had similarly high performance and are strong alternatives for this dataset. In contrast, Decision Trees and Naïve Bayes had the lowest performance making them the least suitable for this analysis.

Table 12

Classification Summary Table

Classifier	Precision	Recall	Specificity	F1-Score	AUC
Decision Tree (No Pruning)	0.65	0.69	0.682	0.66	0.66
Decision Tree (Pre-Pruning)	0.83	0.51	0.89	0.63	0.70
Decision Tree (Post-Pruning)	0.85	0.47	0.91	0.61	0.71

Logistic Regression	0.74	0.68	0.76	0.71	0.80
kNN	0.74	0.68	0.76	0.71	0.80
SVM	0.800	0.590	0.851	0.679	0.80
Naïve Bayes	0.71	0.58	0.75	0.64	0.71
MLP	0.76	0.67	0.79	0.71	0.81

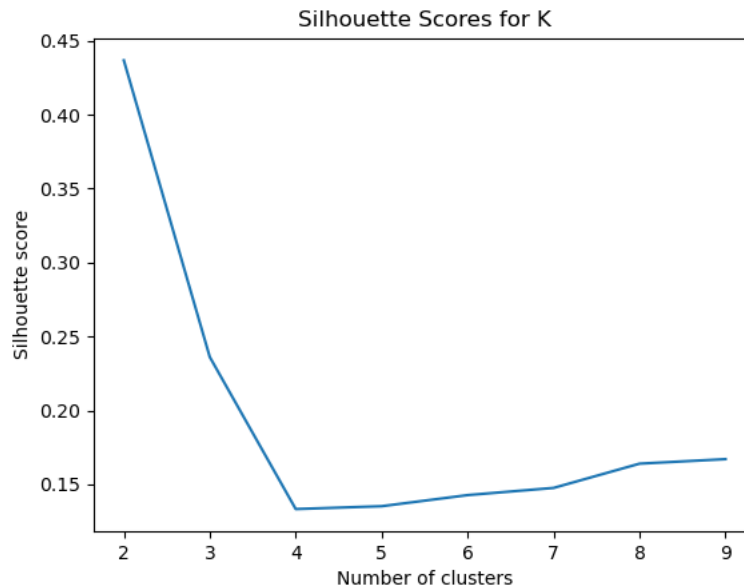
Phase IV: Clustering and Association Rule Mining

Phase IV studied clustering and association rules in the dataset. K-means and DBSCAN were used for clustering, while the Apriori algorithm was used for association rule mining. The scikit-learn and MLxtend libraries were used for clustering and association rule mining, respectively (Pedregosa et al., 2011; Raschka, 2023).

K-means (Clustering)

K-means clustering is an unsupervised method that groups data points based on similarity measures such as Euclidean distance. A key challenge with k-means is determining the optimal number of clusters (k) before applying the algorithm. To address this issue, the silhouette method and the elbow method were both applied in this study to find the best value for k.

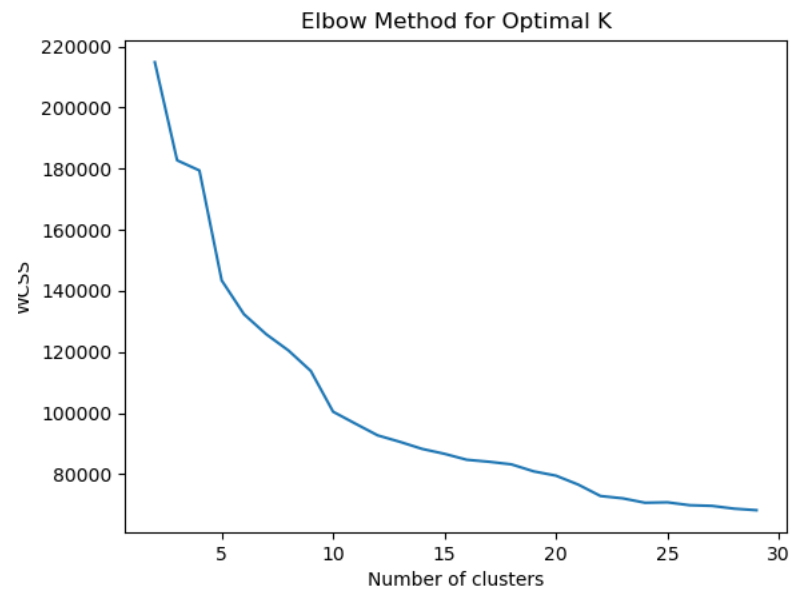
The silhouette score evaluates the quality of the clusters created by k-means. The score ranges from -1 to 1, where 1 indicates well separated clusters, and -1 suggests overlapping clusters. To determine the optimal number of clusters for this dataset, k values in the range of 2 to 10 were tested. The silhouette score for each k value was calculated and the results were plotted as shown in Figure 24. When this method was applied, the optimal k value was identified as 2, since it resulted in the highest silhouette score. However, a silhouette score was still relatively low, meaning that these groups are not well separated.

Figure 24*K-Means Clustering Silhouette Method***Table 13***K-Means Clustering Results (Silhouette Method)*

Cluster	Number of Points in Cluster
0	51,774
1	6,268

The elbow method was also used as another method to find the optimal number of clusters. This method calculates the within-cluster sum of square (WCSS) for different k values. As the number of clusters increases, the WCSS decreases until a certain point and then forms an “elbow.” At this point, adding more clusters does not improve the fit within the clusters. K values between 2 to 30 were evaluated. When this technique was applied the optimal value for k was determined to be 15, based on the “elbow” point shown in Figure 25. Further analysis showed that the clusters have varying sizes ranging from 3 data points to 23,271 data points.

Figure 25*K-Means Clustering Elbow Method*

**Table 14**

K-Means Clustering Results (Elbow Method)

Cluster	Number of Points in Cluster
0	2,698
1	5,203
2	1,420
3	23,271
4	301
5	3
6	9,382
7	4
8	31
9	150
10	15
11	892
12	8,059
13	3,533
14	3,080

DBSCAN (Clustering)

DBSCAN is an unsupervised method for density-based clustering. It creates clusters by separating regions of high density from lower density areas. One of the primary benefits of this algorithm is that it doesn't require the number of clusters to be determined beforehand. There

are two primary parameters for DBSCAN: epsilon (the maximum distance used to consider points as neighbors) and minimum points (the minimum number of points needed to consider a dense region). A good rule of thumb for the minimum points is to use approximately 2x the number of features (Kumar, 2024). In this case the minimum samples were set to 120 and the epsilon was set to five. This configuration resulted in one cluster and 76 noise points, indicating that the structure of this dataset might not be well suited for DBSCAN clustering.

Apriori (Association Rule Mining)

The Apriori algorithm is a method for association rule mining and identifies relationships between items in a dataset. It uses three metrics to evaluate the relationships: support, confidence, and lift. The support shows the frequency of an item (or itemset) where a higher value indicates the itemset is more common. The confidence shows the strength of the association between items. Finally, lift is used to determine if the association is meaningful and stronger than random chance.

The Apriori algorithm was applied to the real estate dataset to uncover association rules within the data. However, additional preprocessing was required to prepare the data for the algorithm. All the numerical features had to be discretized. To do this, numerical features such as bath, bed, size, etc. were split into categories of “low”, “medium”, and “high”. Once they were discretized, the features were transformed using one-hot encoding to make them compatible with the Apriori algorithm.

The Apriori model was configured with a minimum confidence threshold of 0.6 to ensure only strong association rules were retained. Table 15 and 16 illustrate the most significant rules, where Table 15 is ranked by confidence scores, and Table 16 is sorted by lift scores.

Table 15

Apriori Results Sorted by Confidence

Antecedents	Consequents	Antecedent Support	Consequent Support	Support	Confidence	Lift	Leverage	Conviction	Zhang's Metric
(year_2022, month_3, bath category_1, status_for_sale, acre_lot category_2, house_size category_1)	(bed category_1)	0.103	0.751	0.102	0.989	1.316	0.025	21.729	0.268
(year_2022, month_3, status_for_sale, bed category_1, acre_lot category_2, house_size category_1)	(bath category_1)	0.103	0.776	0.102	0.988	1.274	0.022	19.116	0.240
(month_3, bath category_1, status_for_sale, acre_lot category_2, house_size category_1)	(bed category_1)	0.105	0.751	0.104	0.988	1.316	0.025	21.122	0.268
(month_3, status_for_sale, bed category_1, acre_lot category_2, house_size category_1)	(bath category_1)	0.106	0.776	0.104	0.988	1.274	0.022	18.604	0.240
(year_2022, bath category_1, status_for_sale, acre_lot category_2, house_size category_1)	(bed category_1)	0.106	0.751	0.104	0.988	1.315	0.025	19.934	0.268
(year_2022, status_for_sale, bed category_1, acre_lot category_2, house_size category_1)	(bath category_1)	0.106	0.776	0.104	0.988	1.273	0.022	17.975	0.240
(year_2022, month_3, bath category_1, status_for_sale, house_size category_1, price category_0)	(bed category_1)	0.103	0.751	0.102	0.987	1.314	0.024	18.842	0.266
(year_2022, month_3, bed category_1, acre_lot category_2, house_size category_1)	(bath category_1)	0.108	0.776	0.107	0.987	1.272	0.023	16.726	0.240
(month_3, bath category_1, status_for_sale, house_size category_1, price category_0)	(bed category_1)	0.105	0.751	0.104	0.986	1.313	0.025	18.428	0.267
(year_2022, month_3, bath category_1, acre_lot category_2,	(bed category_1)	0.108	0.751	0.107	0.986	1.313	0.025	18.179	0.267

house_size category_1)									
---------------------------	--	--	--	--	--	--	--	--	--

Table 16*Apriori Results Sorted by Lift*

Antecedents	Consequents	Antecedent Support	Consequent Support	Support	Confidence	Lift	Leverage	Conviction	Zhang's Metric
(bath category_1, month_3, price category_0)	(status_for_sale, house_size category_1, year_2022, bed category_1)	0.125	0.204	0.102	0.816	4.004	0.076	4.324	0.857
(bath category_1, month_3, bed category_1, price category_0)	(house_size category_1, year_2022, status_for_sale)	0.117	0.219	0.102	0.868	3.970	0.076	5.939	0.847
(month_3, bed category_1, price category_0)	(bath category_1, house_size category_1, year_2022, status_for_sale)	0.126	0.208	0.102	0.810	3.903	0.076	4.173	0.851
(bath category_1, month_3, acre_lot category_2)	(status_for_sale, house_size category_1, year_2022, bed category_1)	0.129	0.204	0.102	0.792	3.889	0.076	3.835	0.853
(bath category_1, month_3, bed category_1, acre_lot category_2)	(house_size category_1, year_2022, status_for_sale)	0.123	0.219	0.102	0.831	3.801	0.075	4.634	0.840
(bath category_1, month_3, price category_0)	(house_size category_1, year_2022, status_for_sale)	0.125	0.219	0.103	0.827	3.780	0.076	4.510	0.840
(acre_lot category_2, month_3, bed category_1)	(bath category_1, house_size category_1, year_2022, status_for_sale)	0.130	0.208	0.102	0.783	3.774	0.075	3.658	0.845
(bath category_1, year_2022, price category_0, status_for_sale)	(house_size category_1, month_3, bed category_1)	0.115	0.234	0.102	0.882	3.768	0.075	6.513	0.830
(month_3, bed category_1, price category_0)	(house_size category_1, year_2022, status_for_sale)	0.126	0.219	0.103	0.823	3.763	0.076	4.419	0.840
(bath category_1, month_3, acre_lot category_2)	(house_size category_1, year_2022, status_for_sale)	0.129	0.219	0.103	0.802	3.664	0.075	3.937	0.835

It is important to note that the categories were encoded as follows: 0 represents “low”, 1 represents “medium,” and 2 represents “high.” The top rules shown in Table 15 (sorted by confidence) exhibited confidence values near 1.0, meaning the consequent is nearly always true

when the antecedent occurs. The conviction values are all also very high, further emphasizing that there is a strong connection between the antecedent and the consequent. However, the lift values in table 15 are only slightly over 1.0, meaning they may occur by chance and might not be as meaningful. In contrast, Table 16 (sorted by lift) shows lift values above 3.6, indicating these rules are more meaningful. The confidence levels are still relatively high, and range between 0.78 – 0.88, suggesting the consequents are typically true when the antecedent occurs.

Final Recommendations/Conclusion

This project provided valuable insights into the real estate dataset and highlighted the influence of various features on property prices. It emphasized the critical role of preprocessing in machine learning as many steps such as handling missing data, outliers, multicollinearity, scaling the data, and feature encoding were necessary before generating the models. These steps significantly impacted the results of the models. Additionally, the project showed the importance of hyperparameter tuning in achieving high performing models.

The classification process identified MLP classifiers as the top performing model, while logistic regression and kNN also showed strong performance. Future studies could improve classification by exploring additional ensemble methods and stacking. On the other hand, the analysis revealed that the dataset was not well suited for linear regression, due high MSE and low R-squared values. To improve linear regression performance, future work could incorporate additional data to capture market and neighborhood trends to improve the predictive power of the model. Techniques like regularization could also be explored to enhance performance.

For clustering, the project revealed that k-means coupled with the elbow method provides better and more meaningful results compared to DBSCAN for clustering. K-means identified 15 clusters with sizes ranging from 3 to approximately 23,000 data points. Finally, the Apriori algorithm uncovered meaningful relationships within the data. It showed the year 2022,

the month of March and a large lot size often occurs with homes with medium bedroom and bathroom counts, which is reflected by the high lift scores. It also showed that low price and medium bedroom/bathroom counts are typically associated with medium sized houses with the for-sale status.

Overall, these findings can be used by various stakeholders such as real estate investors, agents and policymakers to better understand market trends and make more informed decisions.

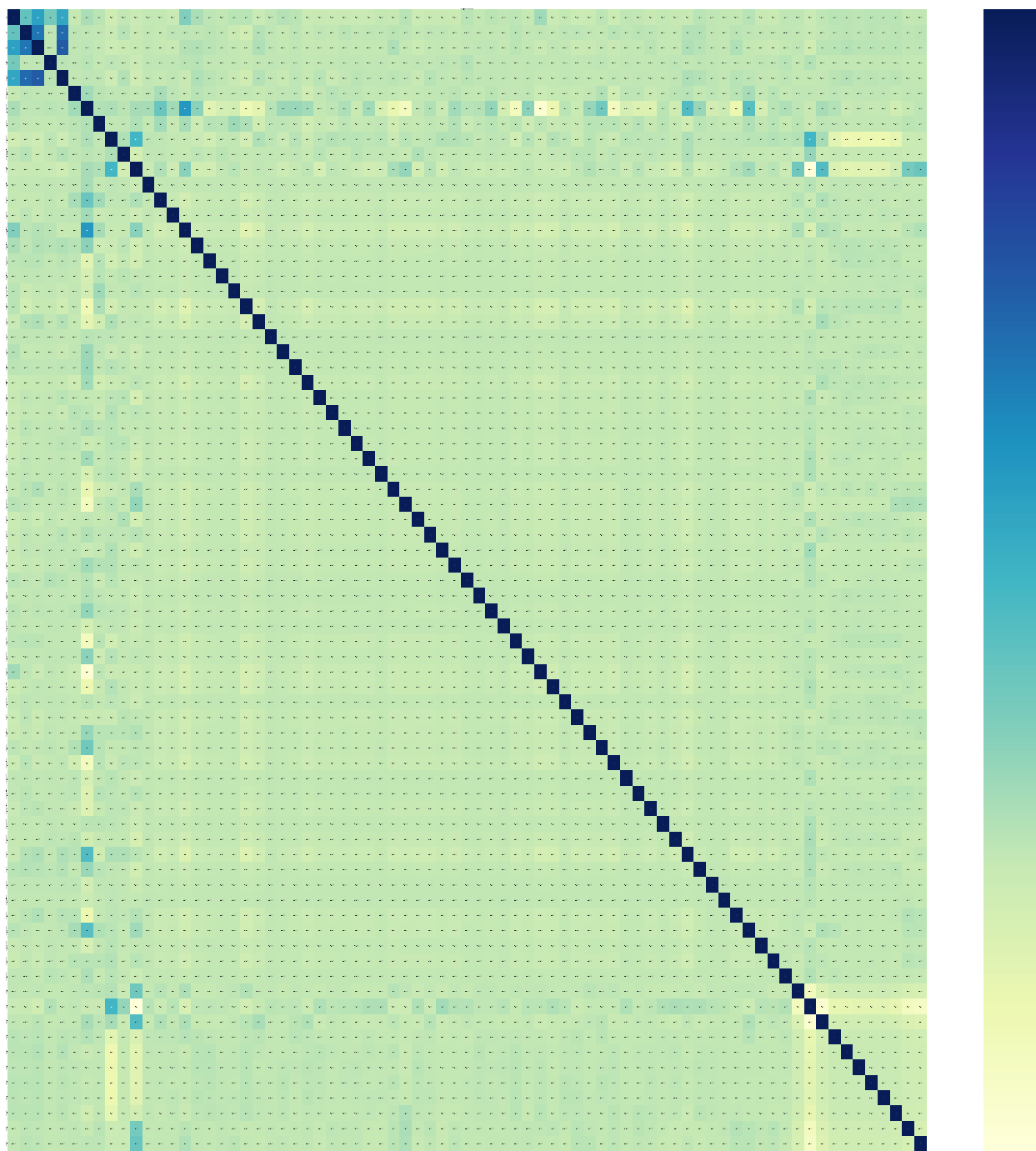
References

- Brownlee, J. (2021, April 26). *How to develop random forest ensembles with XGBoost*. MachineLearningMastery.com. <https://machinelearningmastery.com/random-forest-ensembles-with-xgboost/>
- Kumar, R. (2024, September 29). *A guide to the DBSCAN clustering algorithm*. DataCamp. <https://www.datacamp.com/tutorial/dbscan-clustering-algorithm>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- Raschka, S. (2023). *MLxtend: A Python library for data science and machine learning extensions*. Retrieved from <https://rasbt.github.io/mlxtend/>
- Sakib, A. S. (2024, March 30). *USA Real Estate Dataset*. Kaggle. <https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset>
- Sangole, S. (2024, May 21). *Understanding local outlier factor (LOF) for anomaly detection: A comprehensive guide with python...* Medium. <https://blog.stackademic.com/understanding-local-outlier-factor-lof-for-anomaly-detection-a-comprehensive-guide-with-python-05b2845d92c2>
- Seabold, S., & Perktold, J. (2010). *statsmodels: Econometric and statistical modeling with Python*. In Proceedings of the 9th Python in Science Conference (pp. 57–61).

Appendix

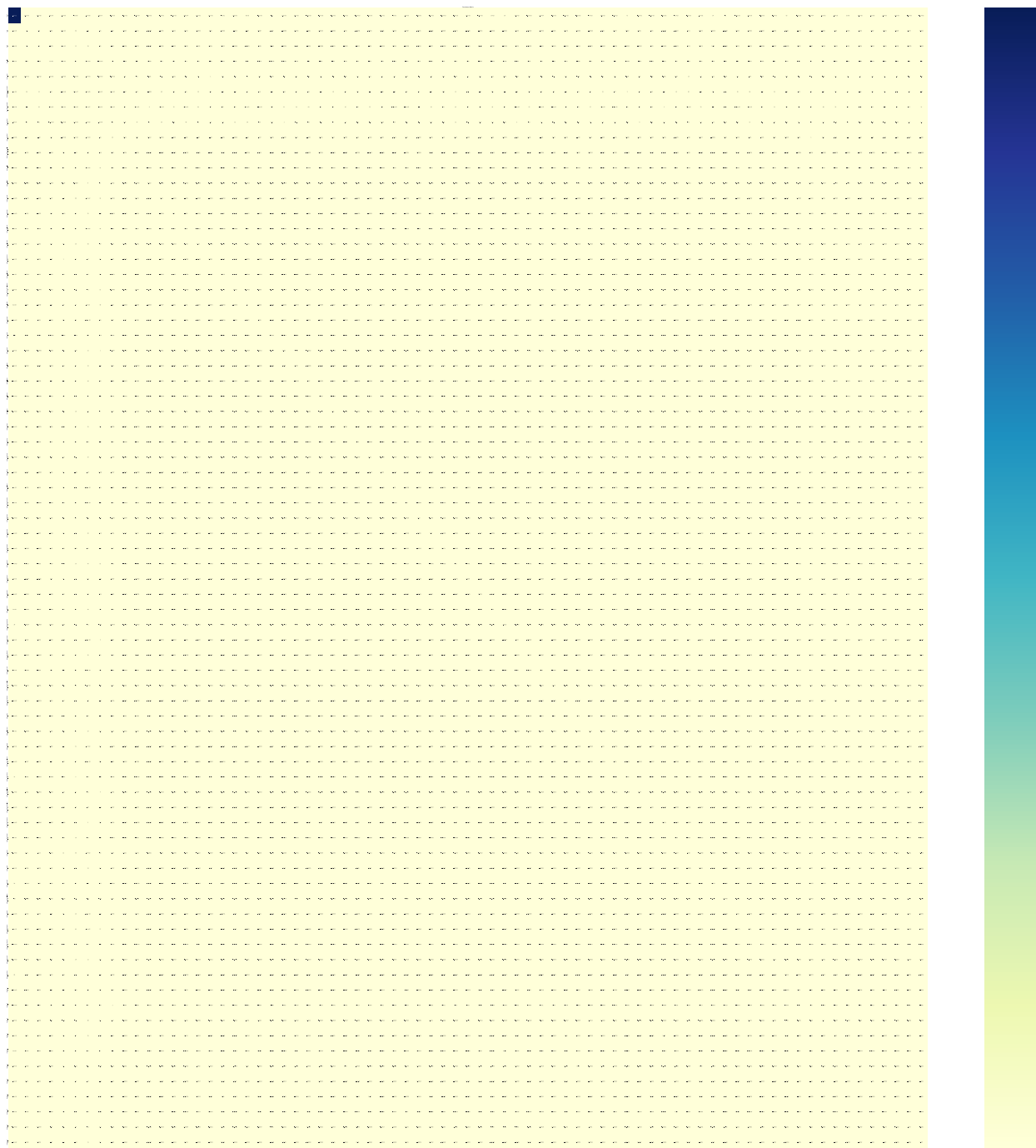
Appendix A

Correlation Heatmap



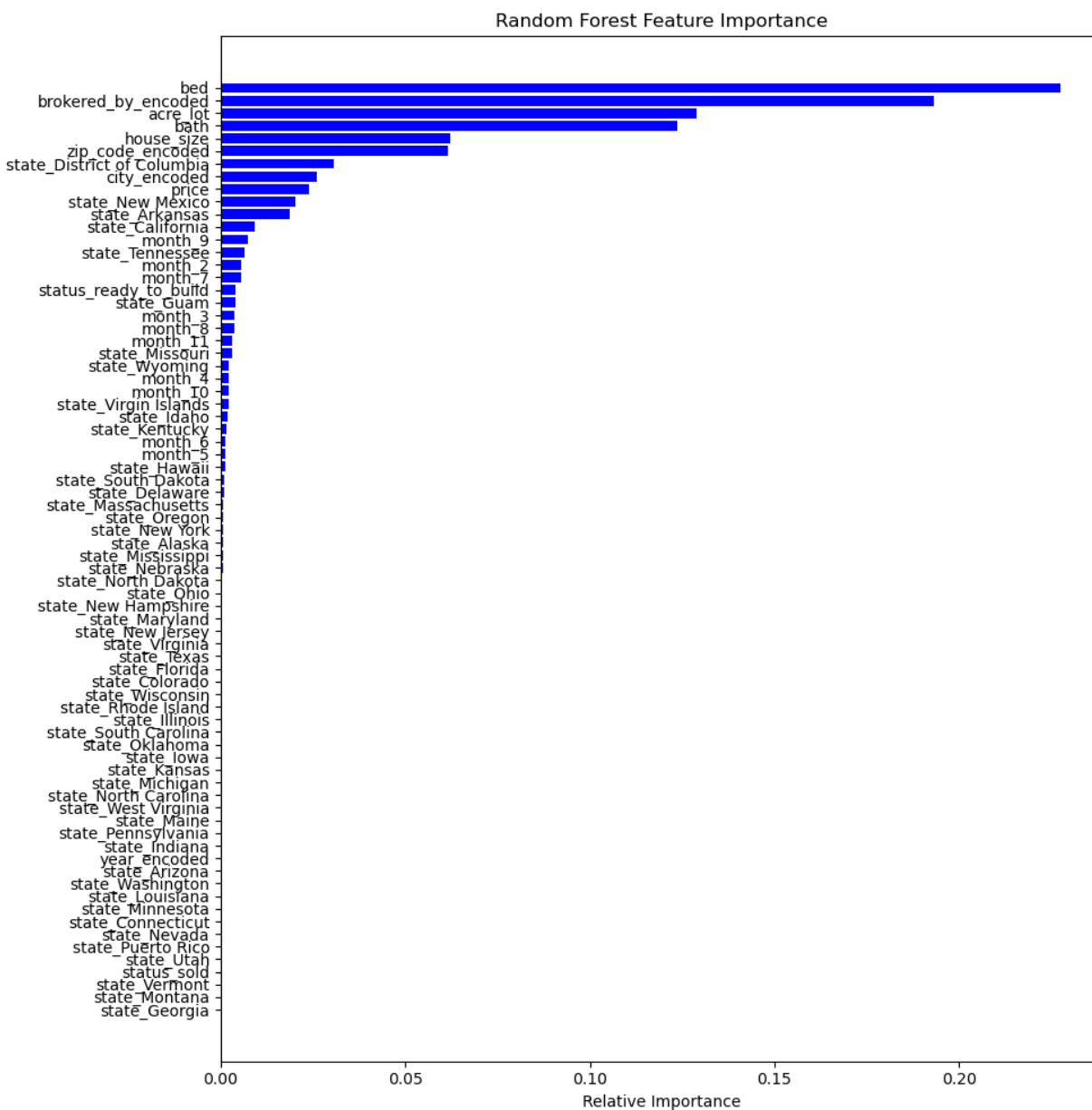
Appendix B

Covariance Heatmap



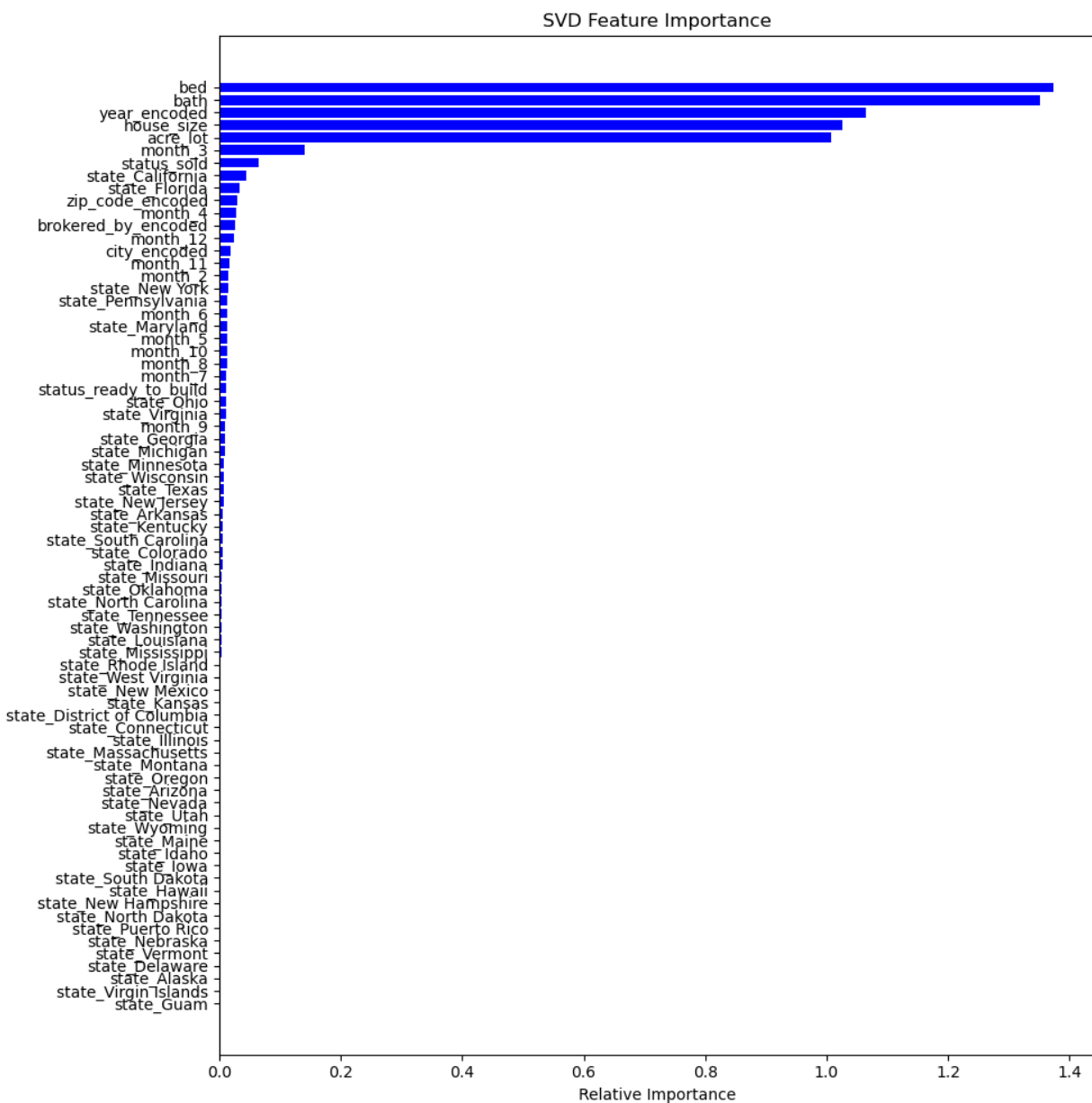
Appendix C

Random Forest Feature Importance



Appendix D

SVD Feature Importance



Appendix E

Supporting Python Code

```

import numpy as np
import pandas as pd
from imblearn.metrics import specificity_score
from imblearn.over_sampling import SMOTE
from matplotlib import pyplot as plt
from xgboost import XGBRFClassifier
from sklearn.metrics import accuracy_score
from prettytable import PrettyTable
from sklearn.cluster import KMeans
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, \
    RocCurveDisplay, precision_score, recall_score, \
    f1_score, mean_squared_error, silhouette_score
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier, LocalOutlierFactor, \
    NearestNeighbors
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from statsmodels.stats.outliers_influence import variance_inflation_factor
import seaborn as sns
from sklearn.model_selection import KFold, train_test_split, GridSearchCV, \
    StratifiedKFold
import statsmodels.api as sm
from tabulate import tabulate
from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.cluster import DBSCAN
import warnings

warnings.filterwarnings("ignore", category=RuntimeWarning)

import logging
logging.getLogger("pycharm_display").setLevel(logging.CRITICAL)

#show all columns
pd.set_option("display.max_columns", None)
#format numbers
pd.set_option('display.float_format', '{:.3f}'.format)

#import data
df = pd.read_csv('/Users/avamohebbi/Documents/MENg/Machine Learning I/Project \
Proposal/realtor-data.zip.csv')
print(df.head())
print(df.shape)

#PHASE 1: EDA

#check/drop duplicates
duplicates = df.duplicated()

```

```

num_duplicates = duplicates.sum()
print("number of duplicates", num_duplicates)

#describe data
print("Dataset Describe:")
print(df.describe())

#check number of values
print('number of unique values:')
print(df.nunique())

#check for missing values in data
print("Missing Data Percentages:")
print()
missing_val_df = (df.isna().sum() / len(df)) * 100
missing_val_df = missing_val_df.reset_index()
missing_val_df.rename(columns={'index': 'Feature', 0: 'Percent Missing'},
inplace=True)
print(missing_val_df)

print(tabulate(missing_val_df, headers='keys', tablefmt='psql'))

missing_val_df.sort_values(by='Percent Missing', inplace=True,
ascending=False)

#create table of
print(tabulate(missing_val_df, headers='keys', tablefmt='psql'))

#impute missing values
df['brokered_by'] = df['brokered_by'].fillna(df['brokered_by'].mode()[0])
df['status'] = df['status'].fillna(df['status'].mode()[0])
df['price'] = df['price'].fillna(df['price'].median())
df['bed'] = df['bed'].fillna(df['bed'].median())
df['bath'] = df['bath'].fillna(df['bath'].median())
df['acre_lot'] = df['acre_lot'].fillna(df['acre_lot'].median())
df['street'] = df['street'].fillna(df['street'].mode()[0])
df['city'] = df['city'].fillna(df['city'].mode()[0])
df['state'] = df['state'].fillna(df['state'].mode()[0])
df['zip_code'] = df['zip_code'].fillna(df['zip_code'].mode()[0])
df['house_size'] = df['house_size'].fillna(df['house_size'].median())
df['prev_sold_date'] =
df['prev_sold_date'].fillna(df['prev_sold_date'].mode()[0])

#print message to confirm data is clean
if df.isnull().values.any() == False:
    print('CONFIRMED: DATASET IS CLEANED')
else:
    print('THERE ARE NULL VALUES REMAINING IN DATASET')

#Use IQR to determine low, medium, high price
Quantile_25 = np.quantile(df['price'], 0.25)
Quantile_75 = np.quantile(df['price'], 0.75)
IQR = Quantile_75 - Quantile_25

def categorize_price(x):
    if x < Quantile_25:
        return 'Low Price'

```

```

    elif x > Quantile_75:
        return 'High Price'
    else:
        return 'Medium Price'

df['price category'] = df['price'].apply(categorize_price)
print('Dataframe after discretization')
print(df.head())

#downsample data
train_sample_df, test_sample_df = train_test_split(df, test_size=0.03,
stratify = df['price category'], random_state=42)
sample_df = test_sample_df.copy()
print('Downsampled Head')
print(sample_df.head())
print(sample_df.head())
print('Downsampled Shape')
print(sample_df.shape)

#drop the price category after downsampling
sample_df.drop(columns=['price category'], inplace=True)

#VISUALIZE DATA HERE

sample_df_num = sample_df[['price', 'bed', 'bath', 'acre_lot', 'house_size']]

# sns.set(rc = {'figure.figsize':(20,20)})
fig, ax = plt.subplots(figsize=(20, 20))
sns.boxplot(data=sample_df_num)
plt.title('Box Plot For Numerical Features')
plt.yscale('log')
plt.show()

state_counts = sample_df['state'].value_counts()
plt.figure(figsize=(12, 8))
sns.countplot(data=sample_df, x='state')
plt.xticks(rotation=90)
plt.title("State Counts")
plt.tight_layout()
plt.show()

status_counts = sample_df['status'].value_counts()
plt.figure(figsize=(12, 8))
sns.countplot(data=sample_df, x='status')
plt.xticks(rotation=90)
plt.title("Status Counts")
plt.tight_layout()
plt.show()

#drop unnecessary features
sample_df.drop(columns=['street'], axis=1, inplace=True)

#list of numeric features
sample_df_numeric = sample_df[['price', 'bed', 'bath', 'acre_lot',
'house_size']]

#list of categorical features

```

```

categorical_features = ['brokered_by', 'status', 'city', 'state', 'zip_code']

#label encoding for categorical features
le_brokered_by = LabelEncoder()
le_zip_code = LabelEncoder()
le_city = LabelEncoder()

sample_df['brokered_by_encoded'] =
le_brokered_by.fit_transform(sample_df['brokered_by'])
sample_df['zip_code_encoded'] =
le_zip_code.fit_transform(sample_df['zip_code'])
sample_df['city_encoded'] = le_city.fit_transform(sample_df['city'])

sample_df['prev_sold_date'] = pd.to_datetime(sample_df['prev_sold_date'])
sample_df['month'] = sample_df['prev_sold_date'].dt.month
sample_df['year'] = sample_df['prev_sold_date'].dt.year

#copy dataframe to use for apriori
sample_df_arm = sample_df.copy()

year_mapping = {1901: 0, 1910: 1, 1952: 2, 1964: 3, 1966: 4, 1967: 5, 1968:
6, 1969: 7, 1970: 8, 1971: 9,
 1972: 10, 1973: 11, 1974: 12, 1975: 13, 1976: 14, 1977: 15, 1978: 16, 1979:
17, 1980: 18,
 1981: 19, 1982: 20, 1983: 21, 1984: 22, 1985: 23, 1986: 24, 1987: 25, 1988:
26, 1989: 27,
 1990: 28, 1991: 29, 1992: 30, 1993: 31, 1994: 32, 1995: 33, 1996: 34, 1997:
35, 1998: 36,
 1999: 37, 2000: 38, 2001: 39, 2002: 40, 2003: 41, 2004: 42, 2005: 43, 2006:
44, 2007: 45,
 2008: 46, 2009: 47, 2010: 48, 2011: 49, 2012: 50, 2013: 51, 2014: 52, 2015:
53, 2016: 54,
 2017: 55, 2018: 56, 2019: 57, 2020: 58, 2021: 59, 2022: 60, 2023: 61}

sample_df["year_encoded"] = sample_df["year"].map(year_mapping)

#one hot encoding for state/status
sample_df = pd.get_dummies(sample_df, columns=['status', 'state', 'month'],
dtype=int, drop_first=True)

print('sample df head after encoding:')
print(sample_df.head())

#drop the original categorical columns
sample_df.drop(columns=['brokered_by', 'city', 'zip_code', 'prev_sold_date',
'year'], inplace=True)

print('sample df columns:', sample_df.columns)

sample_df_scaled = sample_df.copy()

numerical_features_lof = ['bed', 'bath', 'acre_lot', 'house_size']
# numerical_features = ['acre_lot', 'house_size']
scaler_lof = StandardScaler()

sample_df_scaled[numerical_features_lof] =
scaler_lof.fit_transform(sample_df_scaled[numerical_features_lof])

```

```

#nomalize the label encoded data
label_encoded_features_lof = ['zip_code_encoded', 'city_encoded',
                              'brokered_by_encoded']
Label_Encoded_Scaler_lof = MinMaxScaler()
sample_df_scaled[label_encoded_features_lof] =
Label_Encoded_Scaler_lof.fit_transform(sample_df_scaled[label_encoded_features_lof])

#outlier detection with LOF
lof = LocalOutlierFactor(n_neighbors=20, contamination='auto')
y_pred = lof.fit_predict(sample_df_scaled)
sample_df['outlier'] = y_pred
sample_df['LOF_score'] = lof.negative_outlier_factor_
outliers = sample_df[sample_df['LOF_score'] < -2.5]
print(outliers.head())
print('outliers shape', outliers.shape)

sample_df.drop(sample_df[sample_df['LOF_score'] < -2.5].index, inplace=True)
print('outliers removed head', sample_df.head())

#go back and drop the outlier columns
sample_df.drop(columns=['LOF_score', 'outlier'], inplace=True)
print('df with outliers removed head:', sample_df.head())
print('df with outliers removed shape:', sample_df.shape)

#split the data into dependent and independent variables (unscaled data)
X = sample_df.drop('price', axis=1)
y = sample_df['price']

#scale the data
numerical_features = ['bed', 'bath', 'acre_lot', 'house_size']
# numerical_features = ['acre_lot', 'house_size']
scaler = StandardScaler()
X_scaled = X.copy()
X_scaled[numerical_features] =
scaler.fit_transform(X_scaled[numerical_features])

#nomalize the label encoded data
label_encoded_features = ['zip_code_encoded', 'city_encoded',
                          'brokered_by_encoded']
Label_Encoded_Scaler = MinMaxScaler()
X_scaled = X_scaled.copy()
X_scaled[label_encoded_features] =
Label_Encoded_Scaler.fit_transform(X_scaled[label_encoded_features])

print(X_scaled.head())

# # #1: random forest
print('RANDOM FOREST FEATURE SELECTION')
random_forest_model = RandomForestRegressor(random_state=1)
random_forest_model.fit(X, y)
features = sample_df.columns
importances = random_forest_model.feature_importances_
indices = np.argsort(importances)[-77:]
print('Random Forest Indices', [features[i] for i in indices])

```

```

plt.figure(figsize=(10, 10))
plt.title('Random Forest Feature Importance')
plt.barh(range(len(indices)), importances[indices], color='b',
align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.tight_layout()
plt.show()

feature_importance_df = pd.DataFrame({'feature': X.columns, 'importance':
importances.copy()})
feature_importance_df = feature_importance_df.sort_values(by='importance',
ascending=False)
print("Feature Importances Sorted")
print(feature_importance_df.to_string())

not_important_features = []

for index, row in feature_importance_df.iterrows():
    if row['importance'] < 0.001:
        not_important_features.append(row['feature'])

print('not important features:', not_important_features)

#
# #
# # #2:PCA
print("PRINCIPAL COMPONENT ANALYSIS")
#create instance of PCA
pca = PCA(n_components='mle', svd_solver='full')

#computes principle components from scaled data
pca.fit(X_scaled)
X_PCA = pca.transform(X_scaled)
print('explained variance', pca.explained_variance_ratio_)
explained_var_cum = pca.explained_variance_ratio_.cumsum()
num_component_over_95 = np.argmax(explained_var_cum >= 0.95) + 1
print("number of components that explain over 95% of variance:",
num_component_over_95)
#
#Plot PCA
plt.figure(figsize=(20, 10))
plt.plot(np.arange(1, len(np.cumsum(pca.explained_variance_ratio_))+1, 1),
np.cumsum(pca.explained_variance_ratio_))
plt.xticks(np.arange(1, len(np.cumsum(pca.explained_variance_ratio_))+1, 1))
plt.axhline(y=0.95, color='r', linestyle='--')
plt.axvline(x=num_component_over_95, color='b', linestyle='--')
plt.grid()
plt.xlabel('number of components')
plt.ylabel('cumulative explained variance')
plt.title('PCA')
plt.show()

covariance_matrix_pca = pca.get_covariance()
condition_number = np.linalg.cond(covariance_matrix_pca)
print("Condition number:", condition_number)

```

```

# #
# #3: SVD
print('SINGULAR VALUE DECOMPOSITION ANALYSIS')
svd = TruncatedSVD(n_components=5, random_state=42)
svd.fit(X_scaled)
print('svd explained variance ratio', svd.explained_variance_ratio_)
print('svd explained variance ratio sum',
      svd.explained_variance_ratio_.sum())
print('svd singular values', svd.singular_values_)

#get feature importance scores and sort the features by importance
features_SVD = X_scaled.columns
feature_importance = np.sum(np.abs(svd.components_), axis=0)
sorted_indices = np.argsort(feature_importance)[-77:]

plt.figure(figsize=(10, 10))
plt.title('SVD Feature Importance')
plt.barh(range(len(sorted_indices)), feature_importance[sorted_indices],
         color='b', align='center')
plt.yticks(range(len(sorted_indices)), [features_SVD[i] for i in
sorted_indices])
plt.xlabel('Relative Importance')
plt.tight_layout()
plt.show()
#
# # #4: VIF (Variance Inflation Factor)

#create new df for VIF
df_VIF = sample_df.copy()

#drop the target column, and keep independent variables
df_VIF = df_VIF.drop('price', axis=1)

#create a new table to display the VIF outputs
VIF_table = pd.DataFrame()
VIF_table['Feature'] = df_VIF.columns
VIF_table['VIF'] = [variance_inflation_factor(df_VIF.values, i) for i in
range(len(df_VIF.columns))]
VIF_table_sorted = VIF_table.sort_values(by='VIF', ascending=False)
print('VIF Table')
print(VIF_table_sorted.to_string(index=False))

# #covariance heatmap
# plt.figure(figsize=(200,200))
covar_heat_map = sns.heatmap(sample_df.cov(), cmap="YlGnBu", annot=True)
# plt.title("Covariance Matrix")
# plt.savefig("CovarianceMatrix.pdf", format="pdf", bbox_inches="tight")
# plt.show()
#
# #correlation heat map
#
# plt.figure(figsize=(200,200))
corr_heat_map = sns.heatmap(sample_df.corr(), cmap="YlGnBu", annot=True)
# plt.title('Correlation Heatmap')
# plt.savefig("CorrelationMatrix.pdf", format="pdf", bbox_inches="tight")
# plt.show()

```

```

#drop collinear featuresc
sample_df.drop(columns=['zip_code_encoded', 'state_California',
                        'state_Texas', 'state_Washington', 'house_size',
                        'state_Arizona', 'bed', 'year_encoded',
                        'state_Oregon',
                        'month_3', 'state_Illinois', 'state_Florida',
                        'state_Colorado',
                        'state_Oklahoma', 'state_Missouri', 'state_Vermont',
                        'state_Delaware',
                        'state_Puerto Rico', 'state_Hawaii', 'state_North
Dakota', 'state_Wyoming',
                        'state_South Dakota', 'state_Maine', 'state_Nevada',
                        'state_Kansas', 'state_Idaho',
                        'state_Utah', 'state_Rhode Island',
                        'state_Connecticut', 'state_District of Columbia'], inplace=True)

X.drop(columns=['zip_code_encoded', 'state_California',
                'state_Texas', 'state_Washington', 'house_size',
                'state_Arizona', 'bed', 'year_encoded',
                'state_Oregon',
                'month_3', 'state_Illinois', 'state_Florida',
                'state_Colorado',
                'state_Oklahoma', 'state_Missouri', 'state_Vermont',
                'state_Delaware',
                'state_Puerto Rico', 'state_Hawaii', 'state_North
Dakota', 'state_Wyoming',
                'state_South Dakota', 'state_Maine', 'state_Nevada',
                'state_Kansas', 'state_Idaho',
                'state_Utah', 'state_Rhode Island',
                'state_Connecticut', 'state_District of Columbia'], inplace=True)

df_cluster = sample_df.copy()

#Backward Stepwise Regression

#create table for stepwise regression
stepwise_reg_table = PrettyTable(['Step', 'AIC', 'BIC', 'Adjusted R-Square',
                                  'Feature with Max. p-value [DROP]', 'p-value'])
stepwise_reg_table.title = 'Backward Stepwise Regression Table'

#train test split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
                                                    random_state=5805, shuffle=True)

updated_numerical_features = ['acre_lot', 'bath']
updated_label_encoded_features = ['city_encoded', 'brokered_by_encoded']

#scale numerical columns
scaler_reg = StandardScaler()
X_train[updated_numerical_features] =
scaler_reg.fit_transform(X_train[updated_numerical_features])
X_test[updated_numerical_features] =
scaler_reg.fit_transform(X_test[updated_numerical_features])
Label_Encoded_Scaler = MinMaxScaler()

```

```

X_train[updated_label_encoded_features] =
Label_Encoded_Scaler.fit_transform(X_train[updated_label_encoded_features])
X_test[updated_label_encoded_features] =
Label_Encoded_Scaler.fit_transform(X_test[updated_label_encoded_features])

#print train/test sets
print("First 5 rows of the training set (X):")
print(X_train.head())
print('X_train shape:', X_train.shape)

print("First 5 rows of the test set (X):")
print(X_test.head())
print('X_test shape:', X_test.shape)

X_test = sm.add_constant(X_test)

#create function for stepwise regression
def backward_stepwise_reg(training_set, target_set):
    step = 1
    while True:
        training_set = sm.add_constant(training_set)
        modell = sm.OLS(target_set, training_set).fit()
        print('Results 1:', modell.summary())
        stepwise_reg_table.add_row([step, modell.aic, modell.bic,
modell1.rsquared_adj, modell.pvalues.idxmax(),
                                modell.pvalues.loc[modell.pvalues.idxmax()]]])
        if modell.pvalues.loc[modell.pvalues.idxmax()] > 0.05:
            training_set.drop(columns = modell.pvalues.idxmax(), inplace =
True)
            step += 1
        else:
            break

#implmenet and print backward stepwise regression
backward_stepwise_reg(X_train, y_train)
print(stepwise_reg_table)

#develop final regression model
X_train = sm.add_constant(X_train)
final_regression_model = sm.OLS(y_train, X_train).fit()
print(final_regression_model.summary())
final_regression_predictions = final_regression_model.predict(X_test)
final_regression_predictions_train = final_regression_model.predict(X_train)
final_model_mse = mean_squared_error(y_test, final_regression_predictions)

# #create table for final regression model
final_regression_table = PrettyTable(['AIC', 'BIC', 'R-Square', 'Adjusted R-
Square', 'MSE'])
final_regression_table.title = 'Final Regression Model'
final_regression_table.add_row([final_regression_model.aic,
final_regression_model.bic, final_regression_model.rsquared,
final_regression_model.rsquared_adj, final_model_mse])

print(final_regression_table)

#F-Statistic Analysis

```

```

f_stat = final_regression_model.fvalue
f_stat_p_value = final_regression_model.f_pvalue
print('F-Statistic', f_stat)
print('p-value', f_stat_p_value)

if f_stat_p_value < 0.05:
    print('p value for f-statistic is less than 0.05 so reject the null
hypothesis (the model is significant)')
else:
    print('p value for f-statistic is more than 0.05 so fail to reject the
null hypothesis')

# plt.figure(figsize = (10, 10))
plt.title('Final Regression Model [TEST]')
plt.scatter(y_test, final_regression_predictions, s=1, color='blue')

max_val = max(np.max(y_test), np.max(final_regression_predictions))
min_val = min(np.min(y_test), np.min(final_regression_predictions))
plt.xlabel('Actual Value [Test]')
plt.ylabel('Predicted Values')
plt.plot([min_val, max_val], [min_val, max_val], 'r--')
plt.show()

# plt.figure(figsize = (10, 10))
plt.title('Final Regression Model [TRAIN]')
plt.scatter(y_train, final_regression_predictions_train, s=1, color='blue')

max_val_train = max(np.max(y_train),
np.max(final_regression_predictions_train))
min_val_train = min(np.min(y_train),
np.min(final_regression_predictions_train))
plt.xlabel('Actual Value [Train]')
plt.ylabel('Predicted Values')
plt.plot([min_val_train, max_val_train], [min_val_train, max_val_train], 'r--')
plt.show()

#confidence interval analysis
confidence_intervals = final_regression_model.conf_int(alpha = 0.95)
print('Confidence Intervals', confidence_intervals)

final_regression_get_predictions =
final_regression_model.get_prediction(X_test)
prediction_interval = final_regression_get_predictions.conf_int(alpha = 0.05)
prediction_mean = final_regression_get_predictions.predicted_mean

#get lower/upper bound of interval
lower_bound = prediction_interval[:,0]
upper_bound = prediction_interval[:,1]

#flatten
prediction_flat = prediction_mean.flatten()
lower_flat = lower_bound.flatten()

```

```

upper_flat = upper_bound.flatten()

#plot confidence intervals
plt.figure(figsize=(100, 8))
plt.title("Sales Prediction With Confidence Interval")
plt.xlabel("# of Samples")
plt.ylabel("Price")
plt.plot(prediction_flat, label='Predicted Price', color='blue',
linewidth=0.25)
plt.fill_between(range(len(prediction_flat)), lower_flat, upper_flat,
color='c', alpha=0.5, label = 'CI')
plt.grid()
plt.legend()
plt.savefig("ConfidenceInterval.pdf", format="pdf", bbox_inches="tight",
dpi=300)
plt.show()

#classification
# #Use IQR to determine low or high price
median = np.median(sample_df['price'])
Quantile_25 = np.quantile(df['price'], 0.25)
Quantile_75 = np.quantile(df['price'], 0.75)
IQR = Quantile_75 - Quantile_25

def categorize_price(x):
    if x > median:
        return 'High Price'
    else:
        return 'Low Price'

sample_df['price category'] = sample_df['price'].apply(categorize_price)

print(sample_df.head())

sns.countplot(x=sample_df['price category'], data=sample_df)
plt.title('Price Category Distribution')
plt.show()

print('unbalanced data counts', sample_df['price category'].value_counts())

#encode the price category
price_map = {'Low Price': 0, 'High Price': 1}
le = LabelEncoder()
sample_df['price category'] = le.fit_transform(sample_df['price
category']).map(price_map)

X_classification = sample_df.drop(['price', 'price category'], axis=1)
y_classification = sample_df['price category']

#train test split
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(
    X_classification, y_classification, random_state=42, test_size=0.20,
shuffle=True)

#use SMOTE to balance dataset
smote = SMOTE(random_state = 42)

```

```

X_train_class, y_train_class = smote.fit_resample(X_train_class,
y_train_class)
print('Check if balanced:')
print(y_train_class.value_counts())
print('x balanced', X_train_class)
print('y balanced', y_train_class)

#plot balanced data
print('y class value counts', y_train_class.value_counts())

scaler_class = StandardScaler()
X_train_class[updated_numerical_features] =
scaler_class.fit_transform(X_train_class[updated_numerical_features])
X_test_class[updated_numerical_features] =
scaler_class.fit_transform(X_test_class[updated_numerical_features])
Label_Encoded_Scaler_class = MinMaxScaler()
X_train_class[updated_label_encoded_features] =
Label_Encoded_Scaler_class.fit_transform(X_train_class[updated_label_encoded_
features])
X_test_class[updated_label_encoded_features] =
Label_Encoded_Scaler_class.fit_transform(X_test_class[updated_label_encoded_f
eatures])

clf_table = PrettyTable(['Classifier', 'Precision', 'Sensitivity/Recall',
'Specificity', 'F-score'])

#stratified k fold object
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

#Naive Bayes Classifier
print('Naive Bayes')

NB_model = GaussianNB()
param_grid_NB = {'var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6, 1e-5]}

#GridSearch
grid_search_NB = GridSearchCV(estimator=NB_model, param_grid=param_grid_NB,
cv=skf, scoring='accuracy')
grid_search_NB.fit(X_train_class, y_train_class)

# #Best Model
best_NB_model = grid_search_NB.best_estimator_
y_pred_NB = best_NB_model.predict(X_test_class)
print("Best parameters [Naive Bayes]:", grid_search_NB.best_params_)
print("Best cross-validated accuracy [Naive Bayes]:",
grid_search_NB.best_score_)

#accuracy for train/test set
y_pred_train_NB = best_NB_model.predict(X_train_class)
y_test_acc_NB = accuracy_score(y_test_class, y_pred_NB)
y_train_acc_NB = accuracy_score(y_train_class, y_pred_train_NB)
print('accuracy on train data [naive bayes]', y_train_acc_NB)
print('accuracy on test data [naive bayes]', y_test_acc_NB)

# #confusion matrix
cm_NB = confusion_matrix(y_test_class, y_pred_NB)
display_NB = ConfusionMatrixDisplay(confusion_matrix=cm_NB)

```

```

display_NB.plot()
plt.title('Naive Bayes Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_NB = precision_score(y_test_class, y_pred_NB)
print("Naive Bayes Precision:", precision_NB)
recall_NB = recall_score(y_test_class, y_pred_NB)
print("Naive Bayes Recall:", recall_NB)
specificity_NB = specificity_score(y_test_class, y_pred_NB)
print("Naive Bayes Specificity:", specificity_NB )
F1_NB = f1_score(y_test_class, y_pred_NB)
print("Naive Bayes F1:", F1_NB)

#add row to table
clf_table.add_row(["Naive Bayes", precision_NB, recall_NB, specificity_NB,
F1_NB])

#ROC
RocCurveDisplay.from_estimator(best_NB_model, X_test_class, y_test_class)
y_prob_test_NB = best_NB_model.predict_proba(X_test_class)[: , 1]
plt.title('Naive Bayes ROC')
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.legend(loc='lower right')
plt.show()

#
#Decision Tree Classifier
print('Decision Tree')
DT_model = DecisionTreeClassifier(random_state=42)

DT_model.fit(X_train_class, y_train_class)

y_pred_DT = DT_model.predict(X_test_class)

y_pred_train_DT = DT_model.predict(X_train_class)
y_test_acc_DT = accuracy_score(y_test_class, y_pred_DT)
y_train_acc_DT = accuracy_score(y_train_class, y_pred_train_DT)
print('accuracy on train data [DT]', y_train_acc_DT)
print('accuracy on test data [DT]', y_test_acc_DT)

#confusion matrix
# #confusion matrix
cm_DT = confusion_matrix(y_test_class, y_pred_DT)
display_DT = ConfusionMatrixDisplay(confusion_matrix=cm_DT)
display_DT.plot()
plt.title('Decision Tree Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_DT = precision_score(y_test_class, y_pred_DT)
print("Decision Tree Precision:", precision_DT)
recall_DT = recall_score(y_test_class, y_pred_DT)
print("Decision Tree Recall:", recall_DT)
specificity_DT = specificity_score(y_test_class, y_pred_DT)

```

```

print("Decision Tree Specificity:", specificity_DT)
F1_DT = f1_score(y_test_class, y_pred_DT)
print("Decision Tree F1:", F1_DT)
#

#add row to table
clf_table.add_row(["Decision Tree", precision_DT, recall_DT, specificity_DT,
F1_DT])

#ROC
RocCurveDisplay.from_estimator(DT_model, X_test_class, y_test_class)
y_prob_test_DT = DT_model.predict_proba(X_test_class)[:, 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Decision Tree [No Pruning] ROC')
plt.legend(loc='lower right')
plt.show()
#

#DT MODEL PREPRUNING
param_grid_DT_Prepruning = [{'max_depth': [1, 2, 3, 4, 5],
                              'min_samples_split': [20,30,40],
                              'min_samples_leaf': [10,20,30],
                              'criterion':['gini','entropy','log_loss'],
                              'splitter':['best','random'],
                              'max_features':['sqrt','log2']}]

#GridSearch
grid_search_DT_Prepruning = GridSearchCV(DT_model,
                                          param_grid=param_grid_DT_Prepruning,
                                          cv=skf,
                                          scoring='accuracy')

grid_search_DT_Prepruning.fit(X_train_class, y_train_class)

#Best Model
best_DT_model_Prepruning = grid_search_DT_Prepruning.best_estimator_
y_pred_DT_Prepruning = best_DT_model_Prepruning.predict(X_test_class)
print("Best parameters [Decision Tree Prepruning]:",
grid_search_DT_Prepruning.best_params_)
print("Best cross-validated accuracy [Decision Tree Prepruning]:",
grid_search_DT_Prepruning.best_score_)

y_pred_train_DT_Pre = best_DT_model_Prepruning.predict(X_train_class)
y_test_acc_DT_pre = accuracy_score(y_test_class, y_pred_DT_Prepruning)
y_train_acc_DT_pre = accuracy_score(y_train_class, y_pred_train_DT_Pre)
print('accuracy on train data [DT Prepruning]', y_train_acc_DT_pre)
print('accuracy on test data [DT Prepruning]', y_test_acc_DT_pre)

#confusion matrix
# #confusion matrix
cm_DT_Prepruning = confusion_matrix(y_test_class, y_pred_DT_Prepruning)
display_DT_Prepruning =
ConfusionMatrixDisplay(confusion_matrix=cm_DT_Prepruning)
display_DT_Prepruning.plot()
plt.title('Decision Tree Prepruning')
plt.show()

```

```

#print precision, recall, f1
precision_DT_Prepruning = precision_score(y_test_class, y_pred_DT_Prepruning)
print("Decision Tree Precision [Prepruning]:", precision_DT_Prepruning)
recall_DT_Prepruning = recall_score(y_test_class, y_pred_DT_Prepruning)
print("Decision Tree Recall [Prepruning]:", recall_DT_Prepruning)
specificity_DT_Prepruning = specificity_score(y_test_class,
y_pred_DT_Prepruning)
print("Decision Tree Specificity [Prepruning]:", specificity_DT_Prepruning)
F1_DT_Prepruning = f1_score(y_test_class, y_pred_DT_Prepruning)
print("Decision Tree F1:", F1_DT_Prepruning)

#add row to table
clf_table.add_row(["Decision Tree [Prepruning]", precision_DT_Prepruning,
recall_DT_Prepruning,
specificity_DT_Prepruning, F1_DT_Prepruning])

#ROC
RocCurveDisplay.from_estimator(best_DT_model_Prepruning, X_test_class,
y_test_class)
y_prob_test_DT_Prepruning =
best_DT_model_Prepruning.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Decision Tree [Pre Pruning] ROC')
plt.legend(loc='lower right')
plt.show()

# #Decision True Post Pruning
path = DT_model.cost_complexity_pruning_path(X_train_class, y_train_class)
alphas = path['ccp_alphas']
print('alphas', alphas)
alfa = np.linspace(0,1,100)
param_grid_DT_Postpruning = {'ccp_alpha': alfa,
                             'criterion': ['gini', 'entropy'],
                             'splitter': ['best'],
                             'max_features':['sqrt']}

#GridSearch
grid_search_DT_Postpruning = GridSearchCV(DT_model,
param_grid=param_grid_DT_Postpruning,
cv=skf,
scoring='accuracy', n_jobs=-1, verbose=2)

grid_search_DT_Postpruning.fit(X_train_class, y_train_class)

#Best Model
best_DT_model_Postpruning = grid_search_DT_Postpruning.best_estimator_
y_pred_DT_Postpruning = best_DT_model_Postpruning.predict(X_test_class)
print("Best parameters [Decision Tree Postpruning]:",
grid_search_DT_Postpruning.best_params_)
print("Best cross-validated accuracy [Decision Tree Postpruning]:",
grid_search_DT_Postpruning.best_score_)

#accuracy for train/test set
y_pred_train_DT_Postpruning =
best_DT_model_Postpruning.predict(X_train_class)
y_test_acc_DT_Postpruning = accuracy_score(y_test_class,

```

```

y_pred_DT_Postpruning)
y_train_acc_DT_Postpruning = accuracy_score(y_train_class,
y_pred_train_DT_Postpruning)
print('accuracy on train data [decision tree with post-pruning]',
y_train_acc_DT_Postpruning)
print('accuracy on test data [decision tree with post-pruning]',
y_test_acc_DT_Postpruning)

#confusion matrix
# #confusion matrix
cm_DT_Postpruning = confusion_matrix(y_test_class, y_pred_DT_Postpruning)
display_DT_Postpruning =
ConfusionMatrixDisplay(confusion_matrix=cm_DT_Postpruning)
display_DT_Postpruning.plot()
plt.title('Decision Tree Postpruning')
plt.show()

#print precision, recall, f1
precision_DT_Postpruning = precision_score(y_test_class,
y_pred_DT_Postpruning)
print("Decision Tree Precision [Postpruning]:", precision_DT_Postpruning)
recall_DT_Postpruning = recall_score(y_test_class, y_pred_DT_Postpruning)
print("Decision Tree Recall [Postpruning]:", recall_DT_Postpruning)
specificity_DT_Postpruning = specificity_score(y_test_class,
y_pred_DT_Postpruning)
print("Decision Tree Specificity [Postpruning]:", specificity_DT_Postpruning)
F1_DT_Postpruning = f1_score(y_test_class, y_pred_DT_Postpruning)
print("Decision Tree F1 [Postpruning]:", F1_DT_Postpruning)

#add row to table
clf_table.add_row(["Decision Tree[Postpruning]", precision_DT_Postpruning,
recall_DT_Postpruning, specificity_DT_Postpruning, F1_DT_Postpruning])

# #ROC
RocCurveDisplay.from_estimator(best_DT_model_Postpruning, X_test_class,
y_test_class)
y_prob_test_DT_Postpruning =
best_DT_model_Postpruning.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Decision Tree [Post Pruning] ROC')
plt.legend(loc='lower right')
plt.show()

#Logistic Regression
LR_model = LogisticRegression(max_iter=1000, random_state=42)

param_grid_LR = {
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear', 'saga'],
    'C': [1, 10, 100, 1000]
}

LR_model.fit(X_train_class, y_train_class)
y_pred_LR = LR_model.predict(X_test_class)

```

```

LR_model.fit(X_train_class, y_train_class)

#GridSearch
grid_search_LR = GridSearchCV(LR_model,
                               param_grid=param_grid_LR,
                               cv=skf,
                               scoring='accuracy')

grid_search_LR.fit(X_train_class, y_train_class)

#Best Model
best_LR_model = grid_search_LR.best_estimator_
y_pred_LR = best_LR_model.predict(X_test_class)
print("Best parameters[Log Reg]:", grid_search_LR.best_params_)
print("Best cross-validated accuracy[Log Reg]:", grid_search_LR.best_score_)

#accuracy for train/test set
y_pred_train_LR = best_LR_model.predict(X_train_class)
y_test_acc_LR = accuracy_score(y_test_class, y_pred_LR)
y_train_acc_LR = accuracy_score(y_train_class, y_pred_train_LR)
print('accuracy on train data [logistic regression]', y_train_acc_LR)
print('accuracy on test data [logistic regression]', y_test_acc_LR)

cm_LR= confusion_matrix(y_test_class, y_pred_LR)
display_LR = ConfusionMatrixDisplay(confusion_matrix=cm_LR)
display_LR.plot()
plt.title('Logistic Regression')
plt.show()

#print precision, recall, f1
precision_LR = precision_score(y_test_class, y_pred_LR)
print("Logistic Regression Precision:", precision_LR)
recall_LR = recall_score(y_test_class, y_pred_LR)
print("Logistic Regression Recall:", recall_LR)
specificity_LR = specificity_score(y_test_class, y_pred_LR)
print("Logistic Regression Specificity:", specificity_LR)
F1_LR = f1_score(y_test_class, y_pred_LR)
print("Logistic Regression F1:", F1_LR)

#add row to table
clf_table.add_row(["Logistic Regression", precision_LR, recall_LR,
specificity_LR, F1_LR])

#ROC
RocCurveDisplay.from_estimator(best_LR_model, X_test_class, y_test_class)
y_prob_test_LR = best_LR_model.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Logistic Regression ROC')
plt.legend(loc='lower right')
plt.show()

#SVM Classifier
print('Support Vector Machine')

```

```

SVM_model = SVC(probability=False, cache_size=1000, random_state=42)
param_grid_SVM = {
    'kernel': ['linear', 'poly', 'rbf'],
    'degree': [2, 3]
}

# GridSearch
grid_search_SVM = GridSearchCV(
    estimator=SVM_model,
    param_grid=param_grid_SVM,
    cv=skf,
    scoring='accuracy',
    verbose=2,
    n_jobs=-1
)
grid_search_SVM.fit(X_train_class, y_train_class)

# Best Model
best_SVM_model = grid_search_SVM.best_estimator_
y_pred_SVM = best_SVM_model.predict(X_test_class)
print("Best parameters [SVM]:", grid_search_SVM.best_params_)
print("Best cross-validated accuracy [SVM]:", grid_search_SVM.best_score_)

#accuracy for train/test set
y_pred_train_SVM = best_SVM_model.predict(X_train_class)
y_test_acc_SVM = accuracy_score(y_test_class, y_pred_SVM)
y_train_acc_SVM = accuracy_score(y_train_class, y_pred_train_SVM)
print('accuracy on train data [support vector machine]', y_train_acc_SVM)
print('accuracy on test data [support vector machine]', y_test_acc_SVM)

# Confusion matrix
cm_SVM = confusion_matrix(y_test_class, y_pred_SVM)
display_SVM = ConfusionMatrixDisplay(confusion_matrix=cm_SVM)
display_SVM.plot()
plt.title('SVM Confusion Matrix')
plt.show()

# Print precision, recall, f1
precision_SVM = precision_score(y_test_class, y_pred_SVM)
print("SVM Precision:", precision_SVM)
recall_SVM = recall_score(y_test_class, y_pred_SVM)
print("SVM Recall:", recall_SVM)
specificity_SVM = specificity_score(y_test_class, y_pred_SVM)
print("SVM Specificity:", specificity_SVM)
F1_SVM = f1_score(y_test_class, y_pred_SVM)
print("SVM F1:", F1_SVM)

#add row to table
clf_table.add_row(["Support Vector Machine", precision_SVM, recall_SVM,
specificity_SVM, F1_SVM])

# ROC
RocCurveDisplay.from_estimator(best_SVM_model, X_test_class, y_test_class)
# y_prob_test_SVM = best_SVM_model.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('SVM ROC')
plt.legend(loc='lower right')

```

```

plt.show()

error = []

for k in range(1, 40):
    model_KN = KNeighborsClassifier(n_neighbors=k, weights='uniform')
    model_KN.fit(X_train_class, y_train_class)
    prediction_KN_k = model_KN.predict(X_test_class)
    e = np.mean(prediction_KN_k != y_test_class)
    error.append(e)

#plot error rate for each k
plt.plot(range(1, 40), error, marker='o', color = 'blue', linestyle='dashed',
markerfacecolor='red', markeredgecolor='black')
plt.xlabel('K value')
plt.ylabel('Error Rate')
plt.title('Error Rate for Each K Value')
plt.show()

print('kNN Classifier')
kNN_model = KNeighborsClassifier(n_neighbors=19)
param_grid_kNN = {'leaf_size': (20,40,1), 'p': (1,2), 'weights': ('uniform',
'distance'), 'metric': ('minkowski', 'chebyshev')}

#GridSearch
grid_search_kNN = GridSearchCV(estimator=kNN_model,
param_grid=param_grid_kNN, cv=skf, scoring='accuracy', n_jobs=-1, verbose=2)
grid_search_kNN.fit(X_train_class, y_train_class)

# #Best Model
best_kNN_model = grid_search_kNN.best_estimator_
y_pred_kNN = best_kNN_model.predict(X_test_class)
print("Best parameters [kNN]:", grid_search_kNN.best_params_)
print("Best cross-validated accuracy [kNN]:", grid_search_kNN.best_score_)

#accuracy for train/test set
y_pred_train_kNN = best_kNN_model.predict(X_train_class)
y_test_acc_kNN = accuracy_score(y_test_class, y_pred_kNN)
y_train_acc_kNN = accuracy_score(y_train_class, y_pred_train_kNN)
print('accuracy on train data [k-nearest neighbors]', y_train_acc_kNN)
print('accuracy on test data [k-nearest neighbors]', y_test_acc_kNN)

#
# #confusion matrix
cm_kNN = confusion_matrix(y_test_class, y_pred_kNN)
display_kNN = ConfusionMatrixDisplay(confusion_matrix=cm_kNN)
display_kNN.plot()
plt.title('kNN Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_kNN = precision_score(y_test_class, y_pred_kNN)
print("kNN Precision:", precision_kNN)
recall_kNN = recall_score(y_test_class, y_pred_kNN)
print("kNN Recall:", recall_kNN)

```

```

specificity_kNN = specificity_score(y_test_class, y_pred_kNN)
print("kNN Specificity:", specificity_kNN)
F1_kNN = f1_score(y_test_class, y_pred_kNN)
print("kNN F1:", F1_kNN)

#add row to table
clf_table.add_row(["kNN", precision_kNN, recall_kNN, specificity_kNN,
F1_kNN])

#ROC
RocCurveDisplay.from_estimator(best_kNN_model, X_test_class, y_test_class)
# y_prob_test_NB = best_kNN_model.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('kNN ROC')
plt.legend(loc='lower right')
plt.show()

#Neural Network

MLP_model = MLPClassifier(random_state=42)

param_grid_MLP = {
    'hidden_layer_sizes': [(62,), (62, 31), (62, 40), (62, 155), (62, 124),
(62, 40, 20), (62, 124, 62)],
    'solver': ['sgd', 'adam'],
    'activation': ['tanh', 'relu'],
}

MLP_model.fit(X_train_class, y_train_class)
y_pred_LR = MLP_model.predict(X_test_class)
MLP_model.fit(X_train_class, y_train_class)

#GridSearch
grid_search_MLP = GridSearchCV(MLP_model,
                                param_grid=param_grid_MLP,
                                cv=skf,
                                scoring='accuracy', n_jobs=-1, verbose=2)

grid_search_MLP.fit(X_train_class, y_train_class)

#Best Model
best_MLP_model = grid_search_MLP.best_estimator_
y_pred_MLP = best_MLP_model.predict(X_test_class)
print("Best parameters[MLP]:", grid_search_MLP.best_params_)
print("Best cross-validated accuracy[MLP]:", grid_search_MLP.best_score_)

#accuracy for train/test set
y_pred_train_MLP = best_MLP_model.predict(X_train_class)
y_test_acc_MLP = accuracy_score(y_test_class, y_pred_MLP)
y_train_acc_MLP = accuracy_score(y_train_class, y_pred_train_MLP)
print('accuracy on train data [multilayer perceptron]', y_train_acc_MLP)
print('accuracy on test data [multilayer perceptron]', y_test_acc_MLP)

cm_MLP = confusion_matrix(y_test_class, y_pred_MLP)
display_MLP = ConfusionMatrixDisplay(confusion_matrix=cm_MLP)
display_MLP.plot()

```

```

plt.title('MLP Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_MLP = precision_score(y_test_class, y_pred_MLP)
print("MLP Precision:", precision_MLP)
recall_MLP = recall_score(y_test_class, y_pred_MLP)
print("MLP Recall:", recall_MLP)
specificity_MLP = specificity_score(y_test_class, y_pred_MLP)
print("MLP Specificity:", specificity_MLP)
F1_MLP = f1_score(y_test_class, y_pred_MLP)
print("MLP F1:", F1_MLP)

#add row to table
clf_table.add_row(["MLP", precision_MLP, recall_MLP, specificity_MLP,
F1_MLP])

#ROC
RocCurveDisplay.from_estimator(best_MLP_model, X_test_class, y_test_class)
y_prob_test_LR = best_MLP_model.predict_proba(X_test_class)[: , 1]
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('MLP ROC')
plt.legend(loc='lower right')
plt.show()

#random forest [BAGGING]
print('Random Forest [bagging]')
RF_model = RandomForestClassifier(random_state=42)
param_grid_RF = {
    'n_estimators': [25, 50, 100, 150],
    'max_features': ['sqrt', 'log2', None],
    'max_depth': [3, 6, 9],
    'max_leaf_nodes': [3, 6, 9],
}

#GridSearch
grid_search_RF = GridSearchCV(estimator=RF_model,
                             param_grid=param_grid_RF,
                             cv=skf,
                             scoring='accuracy', verbose=2,n_jobs=-1)

grid_search_RF.fit(X_train_class, y_train_class)

#Best Model
best_RF_model = grid_search_RF.best_estimator_
y_pred_RF = best_RF_model.predict(X_test_class)
print("Best parameters[Random Forest - Bagging]:",
grid_search_RF.best_params_)
print("Best cross-validated accuracy[Random Forest - Bagging]:",
grid_search_RF.best_score_)

#confusion matrix
# #confusion matrix
cm_RF = confusion_matrix(y_test_class, y_pred_RF)
display_RF = ConfusionMatrixDisplay(confusion_matrix=cm_RF)
display_RF.plot()

```

```

plt.title('Random Forest [Bagging] Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_RF = precision_score(y_test_class, y_pred_RF)
print("Random Forest Precision:", precision_RF)
recall_RF = recall_score(y_test_class, y_pred_RF)
print("Random Forest Recall:", recall_RF)
specificity_RF = specificity_score(y_test_class, y_pred_RF)
print("Random Forest Specificity:", specificity_RF)
F1_RF = f1_score(y_test_class, y_pred_RF)
print("Random Forest F1:", F1_RF)

#ROC
RocCurveDisplay.from_estimator(best_RF_model, X_test_class, y_test_class)
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Random Forest [Bagging] ROC')
plt.legend(loc='lower right')
plt.show()

#random forest [BOOSTING]
print('Random Forest [BOOSTING]')
RF_model_boost = XGBRFClassifier(random_state=42)
param_grid_RF_boost = {
    'n_estimators': [25, 50, 100, 150],
    'max_depth': [3, 6, 9, 12],
}

#GridSearch
grid_search_RF_boost = GridSearchCV(estimator=RF_model_boost,
                                     param_grid=param_grid_RF_boost,
                                     cv=skf,
                                     scoring='accuracy', verbose=2, n_jobs=-1)

grid_search_RF_boost.fit(X_train_class, y_train_class)

#Best Model
best_RF_model_boost = grid_search_RF_boost.best_estimator_
y_pred_RF_boost = best_RF_model_boost.predict(X_test_class)
print("Best parameters[Random Forest - BOOSTING]:",
      grid_search_RF_boost.best_params_)
print("Best cross-validated accuracy[Random Forest - BOOSTING]:",
      grid_search_RF_boost.best_score_)

#confusion matrix
# #confusion matrix
cm_RF_boost = confusion_matrix(y_test_class, y_pred_RF)
display_RF_boost = ConfusionMatrixDisplay(confusion_matrix=cm_RF_boost)
display_RF_boost.plot()
plt.title('Random Forest [Boosting] Confusion Matrix')
plt.show()

#print precision, recall, f1
precision_RF_boost = precision_score(y_test_class, y_pred_RF_boost)
print("Random Forest Precision:", precision_RF_boost)
recall_RF_boost = recall_score(y_test_class, y_pred_RF_boost)
print("Random Forest Recall:", recall_RF_boost)

```

```

specificity_RF_boost = specificity_score(y_test_class, y_pred_RF_boost)
print("Random Forest Specificity:", specificity_RF_boost)
F1_RF_boost = f1_score(y_test_class, y_pred_RF_boost)
print("Random Forest F1:", F1_RF_boost)

#ROC
RocCurveDisplay.from_estimator(best_RF_model_boost, X_test_class,
y_test_class)
plt.plot([0, 1], [0, 1], '--', color='grey', label='Chance Level (AUC=0.5)')
plt.title('Random Forest [BOOSTING] ROC')
plt.legend(loc='lower right')
plt.show()

print('Final Classification Summary Table')
print(clf_table)

#scale df for clustering
print(df_cluster.head())

#scale the data
numerical_features_cluster = ['bath', 'acre_lot', 'price']
standard_scaler_cluster = StandardScaler()
df_cluster[numerical_features_cluster] =
standard_scaler_cluster.fit_transform(df_cluster[numerical_features_cluster])

#nomalize the label encoded data
label_encoded_features_cluster = ['city_encoded', 'brokered_by_encoded']
label_encoded_scaler_cluster = MinMaxScaler()
df_cluster[label_encoded_features_cluster] =
label_encoded_scaler_cluster.fit_transform(df_cluster[label_encoded_features_
cluster])

print('dataframe after scaling:')
print(df_cluster.head())
print("K MEANS: Silhoutte Score Method")

#find best silhoutte score
silhouette_scores = []
num_clusters = [i for i in range(2,10)]

for i in num_clusters:
    k_means_ss = KMeans(n_clusters=i, random_state=42)
    k_means_ss.fit(df_cluster)
    k_means_ss.fit_predict(df_cluster)
    s_score = silhouette_score(df_cluster, k_means_ss.labels_, metric =
'euclidean', random_state=42)
    print('num clusters', i, ', silhouette score:', s_score)
    silhouette_scores.append(s_score)

#plot the scores
plt.plot(num_clusters, silhouette_scores)
plt.title('Silhouette Scores for K')
plt.xlabel('Number of clusters')
plt.ylabel('Silhouette score')
plt.show()

#get number of points per cluster

```

```

k_means_ss_best = KMeans(n_clusters=2, random_state=42)
k_means_ss_best.fit(df_cluster)
k_means_ss_labels = k_means_ss_best.labels_

unique_values_ss, counts_ss = np.unique(k_means_ss_labels,
return_counts=True)
print('unique values and counts [ss]')
print(unique_values_ss, counts_ss)

print("K MEANS: Elbow Method")
wcss_vals = []

for i in range(2,30):
    k_means_wcss = KMeans(n_clusters=i, random_state=42)
    k_means_wcss.fit(df_cluster)
    k_means_wcss.fit_predict(df_cluster)
    wcss = k_means_wcss.inertia_
    print('num clusters', i, ', wcss:', wcss)
    wcss_vals.append(wcss)

#plot scores
plt.plot(range(2,30), wcss_vals)
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()

#get number of points per cluster
k_means_wcss_best = KMeans(n_clusters=15, random_state=42)
k_means_wcss_best.fit(df_cluster)
k_means_wcss_labels = k_means_wcss_best.labels_

unique_values, counts = np.unique(k_means_wcss_labels, return_counts=True)
print('unique values and counts [wcss]')
print(unique_values, counts)

#DBSCAN clustering
print("DBSCAN Clustering")
DB_neighbors = NearestNeighbors(n_neighbors=120)
neighbors_fitted = DB_neighbors.fit(df_cluster)
distances, indices = neighbors_fitted.kneighbors(df_cluster)

print('distances', distances)
print(indices)

plt.figure(figsize=(20,20))
dist = np.sort(distances[:,119], axis=0)
plt.plot(dist)
plt.show()

db_default = DBSCAN(eps = 5, min_samples = 120).fit(df_cluster)
db_labels = db_default.labels_
num_clusters = len(set(db_labels)) - (1 if -1 in db_labels else 0)
num_noise = list(db_labels).count(-1)

print('Number of clusters:', num_clusters)

```

```

print('Number of noise points:', num_noise)

#Association Rule Mining
sample_df_arm.drop(columns=['prev_sold_date'], inplace=True)

# For bed
Quantile_25_bed = np.quantile(sample_df_arm['bed'], 0.25)
Quantile_75_bed = np.quantile(sample_df_arm['bed'], 0.75)
IQR_bed = Quantile_75_bed - Quantile_25_bed

def categorize_bed(x):
    if x < Quantile_25_bed:
        return 'Low Count'
    elif x > Quantile_75_bed:
        return 'High Count'
    else:
        return 'Medium Count'

sample_df_arm['bed category'] = sample_df_arm['bed'].apply(categorize_bed)

# For bath
Quantile_25_bath = np.quantile(sample_df_arm['bath'], 0.25)
Quantile_75_bath = np.quantile(sample_df_arm['bath'], 0.75)
IQR_bath = Quantile_75_bath - Quantile_25_bath

def categorize_bath(x):
    if x < Quantile_25_bath:
        return 'Low Count'
    elif x > Quantile_75_bath:
        return 'High Count'
    else:
        return 'Medium Count'

sample_df_arm['bath category'] = sample_df_arm['bath'].apply(categorize_bath)

# For acre_lot
Quantile_25_acre_lot = np.quantile(sample_df_arm['acre_lot'], 0.25)
Quantile_75_acre_lot = np.quantile(sample_df_arm['acre_lot'], 0.75)
IQR_acre_lot = Quantile_75_acre_lot - Quantile_25_acre_lot

def categorize_acre_lot(x):
    if x < Quantile_25_acre_lot:
        return 'Low Count'
    elif x > Quantile_75_acre_lot:
        return 'High Count'
    else:
        return 'Medium Count'

sample_df_arm['acre_lot category'] =
sample_df_arm['acre_lot'].apply(categorize_acre_lot)

# For house_size
Quantile_25_house_size = np.quantile(sample_df_arm['house_size'], 0.25)
Quantile_75_house_size = np.quantile(sample_df_arm['house_size'], 0.75)
IQR_house_size = Quantile_75_house_size - Quantile_25_house_size

def categorize_house_size(x):

```

```

    if x < Quantile_25_house_size:
        return 'Low Count'
    elif x > Quantile_75_house_size:
        return 'High Count'
    else:
        return 'Medium Count'

sample_df_arm['house_size category'] =
sample_df_arm['house_size'].apply(categorize_house_size)

# For house_size
Quantile_25_price = np.quantile(sample_df_arm['price'], 0.25)
Quantile_75_price = np.quantile(sample_df_arm['price'], 0.75)
IQR_price = Quantile_75_price - Quantile_25_price

def categorize_price(x):
    if x < Quantile_25_price:
        return 'Low Count'
    elif x > Quantile_75_price:
        return 'High Count'
    else:
        return 'Medium Count'

sample_df_arm['price category'] =
sample_df_arm['price'].apply(categorize_price)

bed_map = {'Low Count': 0, 'Medium Count': 1, 'High Count': 2}
le_bed = LabelEncoder()
sample_df_arm['bed category'] = le_bed.fit_transform(sample_df_arm['bed
category']).map(bed_map)

bath_map = {'Low Count': 0, 'Medium Count': 1, 'High Count': 2}
le_bath = LabelEncoder()
sample_df_arm['bath category'] = le_bath.fit_transform(sample_df_arm['bath
category']).map(bath_map)

lot_map = {'Low Count': 0, 'Medium Count': 1, 'High Count': 2}
le_lot = LabelEncoder()
sample_df_arm['acre_lot category'] =
le_bath.fit_transform(sample_df_arm['acre_lot category']).map(lot_map)

size_map = {'Low Count': 0, 'Medium Count': 1, 'High Count': 2}
le_size = LabelEncoder()
sample_df_arm['house_size category'] =
le_bath.fit_transform(sample_df_arm['house_size category']).map(size_map)

price_map = {'Low Count': 0, 'Medium Count': 1, 'High Count': 2}
le_price = LabelEncoder()
sample_df_arm['price category'] = le_bath.fit_transform(sample_df_arm['price
category']).map(price_map)

sample_df_arm.drop(columns=['brokered_by', 'city_encoded', 'city', 'state',
'zip_code', 'zip_code_encoded', 'brokered_by_encoded', 'price', 'bed',
'bath',
                        'acre_lot', 'house_size'], inplace=True)

```

```

sample_df_arm = pd.get_dummies(sample_df_arm, columns = ['bed category',
' bath category', 'acre_lot category', 'house_size category', 'price
category', 'year', 'month', 'status'], dtype=bool, drop_first=False)

print(sample_df_arm.head())

print('Dataframe after discretization')
print(sample_df_arm.head())

#Apriori
frequent_item_sets = apriori(sample_df_arm, min_support=0.1,
use_colnames=True, verbose=1)
print(frequent_item_sets)
frequent_item_sets_rules = association_rules(frequent_item_sets,
metric='confidence', min_threshold=0.6)

frequent_item_sets_rules1 = frequent_item_sets_rules.sort_values(['lift'],
ascending = [False])
print(frequent_item_sets_rules1.head(10).to_string())

frequent_item_sets_rules2 =
frequent_item_sets_rules.sort_values(['confidence'], ascending = [False])
print(frequent_item_sets_rules2.head(10).to_string())

```

